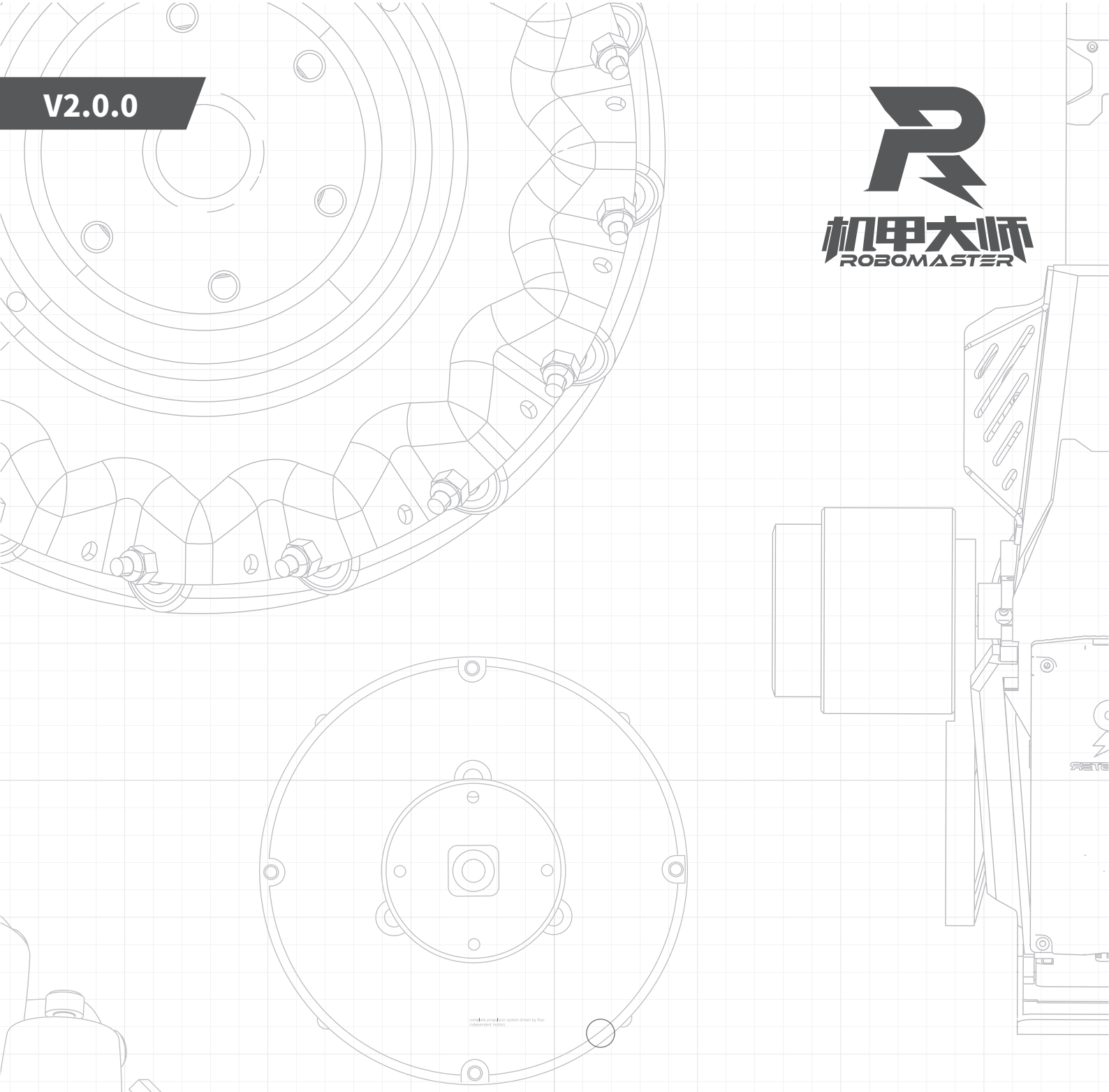
V2.0.0



ROBOMASTER 2026

University Series

Communication Protocol

Prepared by the RoboMaster Organizing Committee

Released in June , 2026

Release Notes

Date	Version	Revision History
2026.06.30	V2.0.0	● Revised command: 0x0003, 0x0120, 0x0201, 0x020C, 0x020D, 0x0301, 0x0A05 ● Revised instructions in the custom client protocol: GameStatus, SentryStatusSync, SentryCtrlCommand, GameStatus, AirSupportStatusSync, AirSupportCommand ● Improved and supplemented multiple descriptions
2026.05.20	V1.3.1	Revised the original MapClickInfoNotify (now split into MapClickInfo and MapClickCmd) in the custom client protocol
2026.03.27	V1.3.0	● Revised command bytes: 0x0101, 0x0305 ● Revised instructions in the custom client protocol: CustomControl, RadarInfoToClient, AirSupportCommand, MapClickInfoNotify, RuneStatusSync, Event, RobotPosition, SentryCtrlCommand, SentryCtrlResult, TechCoreMotionStateSync ● Modified the custom client IP configuration method ● Added custom client proto version information ● Clarified the correspondence between baud rate and physical interface ● Fixed multiple triggering protocol frequencies ● Improved and supplemented multiple descriptions
2026.02.13	V1.2.0	● Revised Command codes 0x0105, 0x0204, 0x0209, 0x020C, 0x020D, 0x0309, and 0x0310. ● Remove Command code 0x0304. ● Revised the original RemoteControl in the custom Client protocol (now split into KeyboardMouseControl and CustomControl), TechCoreMotionStateSync, RobotModuleStatus, RadarInfoToClient, DartCommand, GlobalUnitStatus, RobotModuleStatus, AirSupportStatusSync, DartSelectTargetStatusSync, and RobotPerformanceSelectionCommand. ● Added the CommonCommand Command to the custom Client. ● Fixed multiple issues with descriptions and naming.
2025.12.30	V1.1.0	● Added supplementary details regarding the Radar wireless link.
2025.12.03	V1.0.0	First release

© 2026 DJI All Rights Reserved

Foreword

The scope of this communication protocol for RMUC and RMUL competitions is as follows:

- For entries shared by both RMUC and RMUL (e.g., robot types, mechanics, or battlefield components), the provisions of this protocol apply to both events;
- For entries that apply exclusively to one event shall, by default, not apply to the other.

 © 2026 DJI All Rights Reserved

Table of Contents

Release Notes **1**

Foreword **2**

1 Serial Communication Protocol **4**

1.1 Serial Communication Protocol Format 4

1.2 Instructions for Command Code ID and Standard Data Link 6

1.3 Instructions for Mini-map Interaction Data 33

1.4 Instructions for VTM Link Data 39

1.5 Instructions for Non-data Link Data 40

1.6 Instructions for Radar Wireless Link 42

2 Custom Client Protocol **46**

2.1 Command Overview 46

2.2 Detailed protocol specification 51

Appendix I: CRC check code example **80**

Appendix II: Instruction for ID numbers **84**

Appendix III: Sample communication code for custom client **86**

© 2026 DJI All Rights Reserved

1 Serial Communication Protocol

1.1 Serial Communication Protocol Format

For the section using serial communication, the specifications are as follows: 8 data bits, 1 stop bit, no hardware flow control, and no parity bit. Detailed information is shown in the following table:

Table 1-1 Baud rate

Port	Baud rate
Power Management Modules←→Robot	115200
Custom Controller←→Player's Client	115200
VTM Transmitter←→Robot	921600

Table 1-2 Communication protocol format

frame_header	cmd_id	data	frame_tail
5-byte	2-byte	n-byte	2-byte, CRC16, package-level check

Table 1-3 frame_header format

SOF	data length	seq	CRC8
1 byte	2-byte	1 byte	1 byte

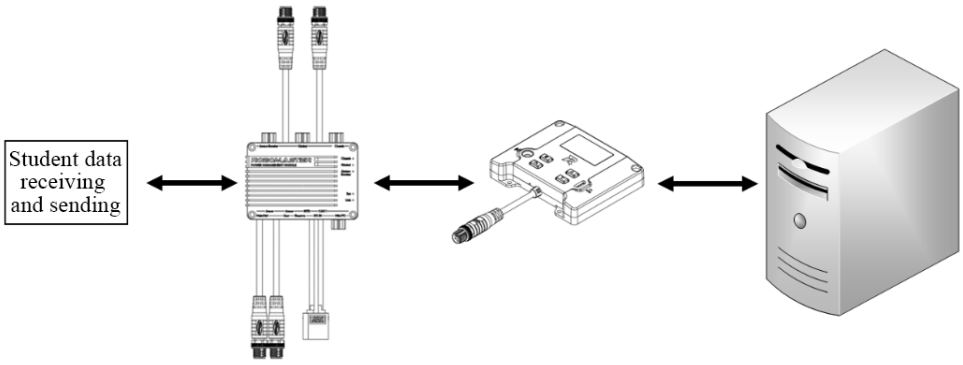
Table 1-4 Detailed definition of frame header

Field	Offset location	Size (bytes)	Detailed description
SOF	0	1	Start byte of the data frame, fixed value is 0xA5
data length	1	2	Length of data in the data frame
seq	3	1	Packet Sequence Number
CRC8	4	1	Frame header CRC8 verification

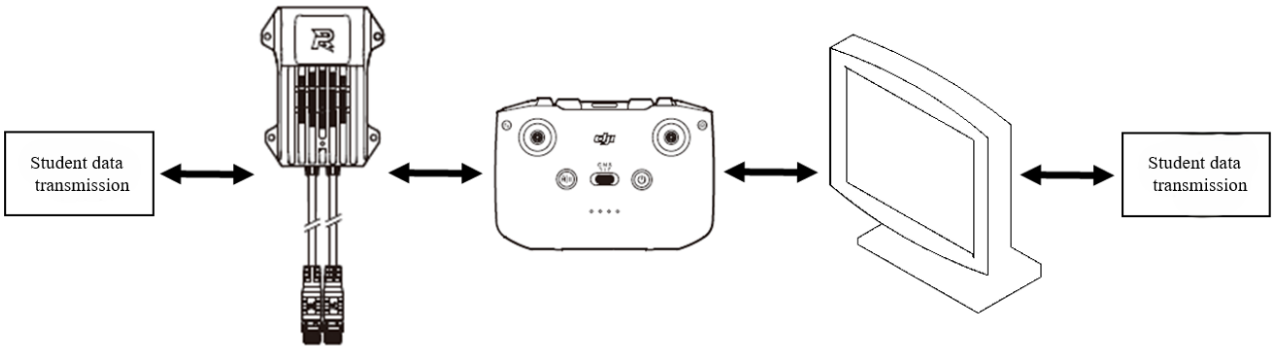
There are three types of Referee System serial data links: the standard link, the VTM link and the Radar wireless link.

 © 2026 DJI All Rights Reserved

- Standard link relays data between the Referee System Server and the Main Control Module, with data sent and received via the Power Management Module's User serial port. The graph is shown below:



- The VTM link relays data between the Referee System Player's Client and the VTM Module, receiving data from the serial port of the VTM Module (Transmitter). See the following graph for details.



- In the Radar wireless link, data is transmitted by the Referee System signal transmitter. The Radar receives the electromagnetic waves and parses the information.

- Under normal operating conditions, Referee System data latency is approximately 130 ms, with a packet loss rate of less than 1%.
- When the battlefield network environment is poor, Referee System data latency is

© 2026 DJI All Rights Reserved

approximately 200 ms, and the packet loss rate is around 3%.

- Measurement data may contain errors and are for reference only.

1.2 Instructions for Command Code ID and Standard Data Link

Table 1-5 Overview of command code IDs

Command code	Data segment length	Description	Sender/Receiver	Data Link
0x0001	11	Competition status data, sent at a fixed frequency of 1Hz.	Server → All robots	Data Link
0x0002	1	Match result data is sent when end of round is Triggered.	Server → All robots	Data Link
0x0003	20	Robots' HP data, sent at a fixed rate of 3Hz.	Server → All robots	Data Link
0x0101	4	Match event data, transmitted at a fixed frequency of 1Hz.	Server → All own side robots	Data Link
0x0104	3	Referee warning data, sent upon trigger by penalty or forfeiture for own side, and transmitted at a frequency of 1Hz at other times.	Server → All robots on the penalized side	Data Link
0x0105	3	Dart launch-related data, sent at a fixed frequency of 1Hz.	Server → All own side robots	Data Link
0x0201	17	Robots performance system data, sent at a fixed frequency of 10Hz.	Main Control Module → respective robot	Data Link
0x0202	14	Real-time chassis buffer energy and barrel heat data, sent at a fixed frequency of 10Hz.	Main Control Module → respective robot	Data Link
0x0203	16	Robots location data, transmitted at a fixed frequency of 1Hz.	Main Control Module → respective robot	Data Link

 © 2026 DJI All Rights Reserved

Command code	Data segment length	Description	Sender/Receiver	Data Link
0x0204	8	Robots' gain and chassis energy data, sent at a fixed frequency of 3Hz.	Server → respective robot	Data Link
0x0206	1	Damage status data, transmitted after damage occurs.	Main Control Module → respective robot	Data Link
0x0207	7	Real-time launch data, sent upon a projectile launched.	Main Control Module → respective robot	Data Link
0x0208	8	Projectile Allowance and Projectile Allowance permitted by remaining coins, transmitted at a fixed frequency of 10Hz.	Server → Own Side Hero, Infantry, Sentry, Drone	Data Link
0x0209	5	Robot RFID module status, sent at a fixed frequency of 3Hz.	Server → Own Side robots equipped with RFID modules	Data Link
0x020A	6	Dart Player's Client Command data, sent at a fixed frequency of 3Hz.	Server → Own Side Dart Robot	Data Link
0x020B	40	Ground robot location data, sent at a fixed frequency of 1Hz.	Server → Own Side Sentry	Data Link
0x020C	2	Radar tracking progress data, transmitted at a fixed frequency of 1Hz.	Server → Own Side Radar Robot	Data Link
0x020D	14	Sentry decision data syncing, transmitted at a fixed rate of 1Hz.	Server → Own Side Sentry	Data Link
0x020E	1	Radar decision data synchronization, transmitted at a fixed frequency of 1Hz.	Server → Own Side Radar Robot	Data Link
0x0301	118	Interaction data between robots, transmission triggered by sender, with a maximum frequency of 30Hz.	-	Data Link

© 2026 DJI All Rights Reserved

Command code	Data segment length	Description	Sender/Receiver	Data Link
0x0302	30	Custom Controller and robot Interaction data; transmission is triggered by the sender, with a maximum frequency of 30Hz.	Custom Controller → robots connected to the player's client for video transmission	VTM Link
0x0303	15	Player's Client Mini-map interaction data, sent when triggered by the Player's Client.	Player's Client click → Server → Own side robots selected by the Sender	Data Link
0x0305	48	Player's Client Mini-map receives Radar data, with a maximum frequency of 5Hz.	Radar → Server → all own side Player's Clients	Data Link
0x0306	8	Custom Controller and Player's Client interaction data, sent when triggered by the sender, with a maximum frequency of 30Hz.	Custom Controller → Player's Client	-
0x0307	103	Player's Client Mini-map receives path data, with a maximum frequency of 1Hz.	Sentry/semi-manually operated robots → respective Player's Client	Data Link
0x0308	34	Player's Client mini-map receives data from robots at a maximum frequency of 3Hz.	Own Side Robots → Own Side Player's Client	Data Link
0x0309	30	The Custom Controller receives data from robots at a maximum frequency of 10Hz.	Own side robots → Custom Controller connected to the corresponding Player's Client	VTM Link
0x0310	300	Data sent from robots to the Custom Client has a maximum frequency of 50Hz.	Own side robots → VTM link → Custom Client connected to the corresponding Player's Client	VTM Link

 © 2026 DJI All Rights Reserved

Command code	Data segment length	Description	Sender/Receiver	Data Link
0x0311	30	Custom Command sent from the custom Client to the Robots, with a maximum frequency of 75Hz.	Custom Client connected to the corresponding Operator's Player's Client → VTM Link → Own Side Robots	VTM Link
0x0A01	24	Position coordinates of opponent robots; continuously send at a frequency of 10Hz.	Signal Transmitter → Radar	Radar Wireless Link
0x0A02	12	HP information of opponent robots; continuously send at a frequency of 10Hz.	Signal Transmitter → Radar	Radar Wireless Link
0x0A03	10	Remaining projectile count of opponent robots; continuously send at a frequency of 10Hz.	Signal Transmitter → Radar	Radar Wireless Link
0x0A04	8	Macro status information of the opponent team; continuously send at a frequency of 10Hz.	Signal Transmitter → Radar	Radar Wireless Link
0x0A05	41	Current buff effects of each opponent robot; continuously send at a frequency of 10Hz.	Signal Transmitter → Radar	Radar Wireless Link
0x0A06	6	Opponent interference wave key; continuously send at a frequency of 10Hz.	Signal Transmitter → Radar	Radar Wireless Link

Table 1-6 0x0001

Byte offset	Size	Description
0	1	• bit 0–3: Competition Type ➢ 1: RoboMaster University Championship (RMUC) ➢ 2: Reserved ➢ 3: Reserved ➢ 4: RoboMaster University League (RMUL) 3v3 Match ➢ 5: RoboMaster University League (RMUL) Infantry Match • bit 4-7: Current match phase ➢ 0: Not started

© 2026 DJI All Rights Reserved

Byte offset	Size	Description
		➢ 1: Preparation Period ➢ 2: 15-Second Referee System Initialization Period ➢ 3: 5-Second Countdown ➢ 4: In Match ➢ 5: Match Settling
1	2	Remaining time in the current phase, in seconds
3	8	UNIX time becomes active once robots are properly connected to the Referee System's NTP Server.

```
typedef _packed struct
{
  uint8_t game_type : 4;
  uint8_t game_progress : 4;
  uint16_t stage_remain_time;
  uint64_t SyncTimeStamp;
}game_status_t;
```

Table 1-7 0x0002

Byte offset	Size	Description
0	1	● 0: Draw ● 1: Red Team wins ● 2: Blue Team wins

```
typedef _packed struct
{
  uint8_t winner;
}game_result_t;
```

Table 1-8 0x0003

Byte offset	Size	Description
0	2	Own Side Hero No.1 HP; if this robot is not fielded or is ejected, its HP is 0. The same applies below.
2	2	Own Side Engineer No.2 HP
4	2	Own Side Infantry No.3 HP
6	2	Own Side Infantry No.4 HP
8	2	The difference between own side total damage and the opponent total damage

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
10	2	Own Side Sentry No.7 HP
12	2	Own Side Outpost HP
14	2	Own Side Base HP
16	2	Opponent Outpost HP
18	2	Opponent Base HP

```
typedef _packed struct
{
  uint16_t ally_1_robot_HP;
  uint16_t ally_2_robot_HP;
  uint16_t ally_3_robot_HP;
  uint16_t ally_4_robot_HP;
  int16_t damage_differenceuint16_t;
  uint16_t ally_7_robot_HP;
  uint16_t ally_outpost_HP;
  uint16_t ally_base_HP;
  uint16_t enemy_outpost_HP;
  uint16_t enemy_base_HP;
} game_robot_HP_t;
```

Table 1-9 0x0101

Byte offset	Size	Description
0	4	0: Not occupied / not activated 1: Occupied / activated ● bit 0-2: ➢ bit 0: Occupation status of own side Resupply Zone; 1 means occupied. ➢ bit 1: Reserved. ➢ bit 2: Occupation status of own side Resupply Zone; 1 indicates occupied. (applicable to RMUL only) ● bit 3-6: Own Side Power Rune status ➢ bit 3-4: Activation status of Own Side Small Power Rune — 0 means not activated, 1 means activated, and 2 means activating. ➢ bit 5-6: Activation status of Own Side Large Power Rune; 0 means not activated, 1 means activated, and 2 means activating.

© 2026 DJI All Rights Reserved

Byte offset	Size	Description
		• bit 7-8: Occupation status of Own Side's central elevated ground, 1 means Occupied by Own Side, 2 means Occupied by Opponent. • bit 9-10: Occupation status of Own Side Trapezoid-Shaped Elevated Ground; 1 indicates occupied. • bit 11-19: The time when the Opponent's Dart last hit Own Side's Outpost or Base (0-420, with an initial value of 0 at the start of the Round). • bit 20-22: The specific goal where the Opponent's Dart last hit Own Side's Outpost or Base. The default value at the start of the Round is 0. 1 means the Outpost was hit, 2 means the fixed target in the Base was hit, 3 means a random fixed target in the Base was hit, 4 means a random moving target in the Base was hit, and 5 means the moving terminal target in the Base was hit. • bit 23-24: Occupation status of the Central Buff Point—0 means not occupied, 1 means occupied by Own Side, 2 means occupied by Opponent, and 3 means occupied by both sides. (Only applicable to RMUL) • bit 25-26: Occupation status of Own Side Fortress Buff Point: 0 means not occupied, 1 means occupied by Own Side, 2 means occupied by Opponent, and 3 means occupied by both sides. • bit 27-28: Occupation status of Own Side Outpost Buff Point — 0 means not occupied, 1 means occupied by Own Side, 2 means occupied by Opponent. • bit 29: Occupation status of Own Side Base Buff Point; 1 indicates occupied. • bit 30-31: Reserved.

```
typedef _packed struct
{
  uint32_t event_data;
}event_data_t;
```

Table 1-10 0x0104

Byte offset	Size	Description
0	1	Own Side's most recent penalty level: • 1: Dual Yellow Card • 2: Yellow Card • 3: Red Card • 4: Forfeiture
1	1	• The offending robot ID for Own Side's most recent penalty. (For example, the ID for Red 1 is 1, and for Blue 1 is 101)

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
		● For Forfeiture and when both sides receive a Yellow Card, this value is 0.
2	1	The number of violations at the corresponding Penalty Level committed by the Offending Robot in Own Side's most recent penalty. (Defaults to 0 at the beginning of the round)

```
typedef _packed struct
{
  uint8_t level;
  uint8_t offending_robot_id;
  uint8_t count;
}referee_warning_t;
```

Table 1-11 0x0105

Byte offset	Size	Description
0	1	Own Side Dart launch remaining time, uint: seconds
1	2	● bit 0-2: The most recent Own Side Dart strike destination. At the start of the round, the default value is 0; 1 indicates a strike on the Outpost; 2 indicates a strike on the Base fixed target; 3 indicates a strike on the Base random fixed target; 4 indicates a strike on the Base random moving target; and 5 indicates a strike on the Base ternimal moving target. ● bit 3–5: The opponent's most recently struck target, total strike count, default is 0 at the beginning of the round, with a maximum of 4. ● bit 6-8: The strike destination currently selected for the dart: 0 if at the beginning of the round or when unselected/selected Outpost, 1 for selected Base fixed target, 2 for selected Base random fixed target, 3 for selected Base random moving target, and 4 for selected Base terminal moving target. ● bit 9-15: Reserved

```
typedef _packed struct
{
  uint8_t dart_remaining_time;
  uint16_t dart_info;
}dart_info_t;
```

© 2026 DJI All Rights Reserved

Table 1-12 0x0201

Byte offset	Size	Description
0	1	Robot ID
1	1	Robot level
2	2	Current roboy HP
4	2	Maximum robot HP
6	2	Robot barrel heat cooling value per second
8	2	Maximum barrel heat
10	2	Maximum chassis power
12	4	Maximum initial launching speed
16	1	Power Management Module output status: ● bit 0: Gimbal port output, 0 indicates no output, 1 indicates 24V output ● bit 1: Chassis port output, 0 means no output, 1 means 24V output ● bit 2: Shooter port output, 0 indicates no output, 1 indicates 24V output

```
typedef _packed struct
{
  uint8_t robot_id;
  uint8_t robot_level;
  uint16_t current_HP;
  uint16_t maximum_HP;
  uint16_t shooter_barrel_cooling_value;
  uint16_t shooter_barrel_heat_limit;
  uint16_t chassis_power_limit;
  float bullet_speed_limit;
  uint8_t power_management_gimbal_output : 1;
  uint8_t power_management_chassis_output : 1;
  uint8_t power_management_shooter_output : 1;
}robot_status_t;
```

 © 2026 DJI All Rights Reserved

Table 1-13 0x0202

Byte offset	Size	Description
0	2	Reserved
2	2	Reserved
4	4	Reserved
8	2	Buffer Energy (uint: J)
10	2	Shooting heat of the 17 mm Launching Mechanism
12	2	Barrel heat of the 42mm Launching Mechanism

```
typedef _packed struct
{
  uint16_t reserved;
  uint16_t reserved;
  float reserved;
  uint16_t buffer_energy;
  uint16_t shooter_17mm_barrel_heat;
  uint16_t shooter_42mm_barrel_heat;
}power_heat_data_t;
```

Table 1-14 0x0203

Byte offset	Size	Description
0	4	Current robot's x-coordinate, uint: m
4	4	Current robot's y-coordinate, uint: m
8	4	Orientation of the robot's Speed Monitor Module, uint: degree. Due north is 0 degree.

```
typedef _packed struct
{
  float x;
  float y;
  float angle;
}robot_pos_t;
```

Table 1-15 0x0204

Byte offset	Size	Description
0	1	HP Recovery Buff (percentage; a value of 10 indicates that 10% of Maximum HP is restored per second)

© 2026 DJI All Rights Reserved

Byte offset	Size	Description
1	2	Specific value of barrel heat cooling buff (direct value; a value of x means cooling rate increases by x per second)
3	1	Defense Buff (percentage; a value of 50 indicates a 50% Defense Buff)
4	1	Negative Defense Buff (percentage; a value of 30 indicates a -30% Defense Buff)
5	2	Attack Buff (percentage; a value of 50 means a 50% Attack Buff)
7	1	bit 0–6: Feedback of remaining energy value for robots, represented as a hexadecimal proportion of the remaining energy. Robots are considered to start with 100% energy. ● bit 0: Set to 1 when remaining energy is ≥ 125%; otherwise, set to 0. ● bit 1: When remaining energy is ≥ 100%, the value is 1; otherwise, the value is 0. ● bit 2: Set to 1 when remaining energy is ≥ 50%, otherwise set to 0. ● bit 3: Set to 1 when remaining energy is ≥ 30%, otherwise set to 0. ● bit 4: Set to 1 when remaining energy is at least 15%, otherwise set to 0. ● bit 5: Set to 1 when remaining energy is ≥ 5%; otherwise, set to 0. ● bit 6: Set to 1 when remaining energy is ≥ 1%; otherwise, set to 0.

```
typedef _packed struct
{
  uint8_t recovery_buff;
  uint16_t cooling_buff;
  uint8_t defense_buff;
  uint8_t vulnerability_buff;
  uint16_t attack_buff;
  uint8_t remaining_energy;
}buff_t;
```

Table 1-16 0x0206

Byte offset	Size	Description
0	1	● bit 0-3: When the reason for HP Loss is that the projectile attack, collision, or disconnection, the value of these 4 bits represents the ID number of the Armor Module or Speed Monitor Module; if HP Loss is caused by miscellaneous reasons, the value is 0. ● bit 4-7: HP variation type ➢ 0: HP loss caused by Armor Module being attacked by a projectile ➢ 1: HP loss caused by a disconnected Armor Module or Supercapacitor Management Module

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
		➢ 5: HP loss occurs when the Armor Module suffers collision.

```
typedef _packed struct
{
  uint8_t armor_id : 4;
  uint8_t HP_deduction_reason : 4;
}hurt_data_t;
```

The injury status for 0x0206 is determined locally by the Referee System and sent immediately. However, whether the corresponding damage is actually incurred depends on the rules and regulations. Please refer to the Server's final determination.

Table 1-17 0x0207

Byte offset	Size	Description
0	1	Projectile type: ● bit 1: 17mm Projectile ● bit 2: 42mm Projectile
1	1	Launching Mechanism ID: ● 1: Launching Mechanism (17mm) ● 2: Reserved ● 3: Launching Mechanism (42mm)
2	1	Projectile firing rate (uint: Hz)
3	4	Projectile speed (uint: m/s)

```
typedef _packed struct
{
  uint8_t projectile_type;
  uint8_t shooter_number;
  uint8_t launching_frequency;
  float projectile_speed;
}shoot_data_t;
```

Table 1-18 0x0208

Byte offset	Size	Description
0	2	Projectile allowance of 17mm projectiles
2	2	42mm projectile allowance
4	2	Remaining Gold Coin amount

© 2026 DJI All Rights Reserved

Byte offset	Size	Description
6	2	Reserved 17mm Projectile Allowance provided by Fortress Buff Point; This value is not related to whether the robot actually occupies the Fortress.

```
typedef _packed struct
{
  uint16_t projectile_allowance_17mm;
  uint16_t projectile_allowance_42mm;
  uint16_t remaining_gold_coin;
  uint16_t projectile_allowance_fortress;
}projectile_allowance_t;
```

Table 1-19 0x0209

Byte offset	Size	Description
0	4	The meaning of bit value 1/0: whether the Buff Point RFID card has been detected ● bit 0: Own Side Base Buff Point ● bit 1: Own Side Central Elevated Ground Buff Point ● bit 2: Opponent Central Elevated Ground Buff Point ● bit 3: Own Side Trapezoid-Shaped Elevated Ground Buff Point ● bit 4: Opponent Trapezoid-Shaped Elevated Ground Buff Point ● bit 5: Own Side Terrain Cross Buff Point (Launch Ramp) (near Own Side ramp, before the ramp) ● bit 6: Own Side Terrain Crossing Buff Point (Launch Ramp) (after the launch ramp on Own Side) ● bit 7: Opponent Terrain Crossing Buff Point (Launch Ramp) (Before Opponent-Side Launch Ramp) ● bit 8: Opponent Terrain Traversal Buff Point (Launch Ramp) (after the launch ramp on Opponent side) ● bit 9: Own Side Terrain Traversal Buff Point (below Trapezoid-Shaped Elevated Ground center) ● bit 10: Own Side Terrain Crossing Buff Point (above the central elevated ground) ● bit 11: Opponent Terrain Crossover Buff Point (below the central elevated ground) ● bit 12: Opponent Terrain Crossing Buff Point (Above Central Elevated Ground) ● bit 13: Own Side Terrain Crossing Buff Point (below Road) ● bit 14: Own Side Terrain Crossing Buff Point (above Road) ● bit 15: Opponent Terrain Crossing Buff Point (below Road)

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
		• bit 16: Opponent Terrain Crossing Buff Point (above Road) • bit 17: Own Side Fortress Buff Point • bit 18: Own Side Outpost Buff Point • bit 19: Own Side Resupply Zone not overlapping with Resource Zone/RMUL Resupply Zone • bit 20: Own Side Resupply Zone overlapping with Resource Zone • bit 21: Own Side Assembly Buff Point • bit 22: Opponent Assembly Buff Point • bit 23: Central Buff Point (only applies to RMUL) • bit 24: Opponent Fortress Buff Point • bit 25: Opponent Outpost Buff Point • bit 26: Own Side Terrain Crossing Buff Point (tunnel) (near the lower area of Own Side Road Zone) • bit 27: Own Side Terrain Crossing Buff Point (tunnel) (near the middle of Own Side Road Zone) • bit 28: Own Side Terrain Crossing Buff Point (tunnel) (near the upper area of Own Side Road Zone) • bit 29: Own Side Terrain Crossing Buff Point (tunnel) (near the lower part of Own Side Trapezoid-Shaped Elevated Ground) • bit 30: Own Side Terrain Crossing Buff Point (tunnel) (near the middle of Own Side Trapezoid-Shaped Elevated Ground) • bit 31: Own Side Terrain Crossing Buff Point (tunnel) (near the higher area of Own Side Trapezoid-Shaped Elevated Ground)
4	1	• bit 0: Opponent Terrain Crossing Buff Point (tunnel) (near the lower side of the Opponent Road) • bit 1: Opponent Terrain Crossing Buff Point (tunnel) (near the middle of the Opponent's Road side) • bit 2: Opponent Terrain Crossing Buff Point (tunnel) (upper area near the Opponent Road side) • bit 3: Opponent Terrain Crossing Buff Point (tunnel) (near the lower area of the Opponent Trapezoid-Shaped Elevated Ground) • bit 4: Opponent Terrain Crossing Buff Point (tunnel) (near the middle of the Opponent Trapezoid-Shaped Elevated Ground) • bit 5: Opponent Terrain Crossing Buff Point (tunnel) (near the higher area of the Opponent Trapezoid-Shaped Elevated Ground)

© 2026 DJI All Rights Reserved

Note: All RFID cards are only active during matches. Outside of matches, even if the corresponding RFID card is detected, the associated value is still 0.

```
typedef _packed struct
{
  uint32_t rfid_status;
  uint8_t rfid_status_2;
}rfid_status_t;
```

Table 1-20 0x020A

Byte offset	Size	Description
0	1	Current status of the Dart Launching Station: ● 1: Off ● 2: Opening or closing ● 0: Already enabled
1	1	Reserved
2	2	Remaining competition time when switching strike target, measured in seconds; defaults to 0 if no switching action is performed.
4	2	Remaining time in the match when the operator last confirms the launch command, in seconds. The initial value is 0.

```
typedef _packed struct
{
  uint8_t dart_launch_opening_status;
  uint8_t reserved;
  uint16_t target_change_time;
  uint16_t latest_launch_cmd_time;
}dart_client_cmd_t;
```

Table 1-21 0x020B

Byte offset	Size	Description
0	4	Own Side Hero Location X-axis coordinate, uint: m
4	4	Own Side Hero Location Y-axis coordinate, uint: m
8	4	Own Side Engineer Location X-axis coordinate, uint: m
12	4	Own Side Engineer Location Y-axis coordinate, uint: m
16	4	Own Side Infantry No. 3 Location X-axis coordinate, uint: m
20	4	Own Side Infantry No. 3 Location Y-axis coordinate, uint: m
24	4	Own Side Infantry No. 4 Location X-axis coordinate, uint: m

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
28	4	Own Side Infantry No. 4 Location Y-axis, uint: m
32	4	Reserved
36	4	Reserved

The intersection of the perimeter wall near the Red Team projectile supplier serves as the origin of the coordinate system. The X-axis extends along the long side of the match field toward the Blue Team, while the Y-axis extends along the short side toward the Red Team landing pad.

```
typedef _packed struct
{
  float hero_x;
  float hero_y;
  float engineer_x;
  float engineer_y;
  float infantry_3_x;
  float infantry_3_y;
  float infantry_4_x;
  float infantry_4_y;
  float reserved;
  float reserved;
}ground_robot_position_t;
```

Table 1-22 0x020C

Byte offset	Size	Description	Remarks
0	2	● bit 0: Vulnerability status of Opponent Hero 1 ● bit 1: Vulnerability status of Opponent Engineer No.2 ● bit 2: Vulnerability status of Opponent Infantry No.3 ● bit 3: Vulnerability status of Opponent Infantry No.4 ● bit 4: Special marking status of Opponent Drone ● bit 5: Vulnerability status of Opponent Sentry ● bit 6: Tracking status for Own Side Hero No.1	Opponent Robots: Transmit “1” when the corresponding robot’s Tracking Progress is greater than or equal to 100, and “0” when the Tracking Progress is less than 100. Own Side Robots: When the corresponding robot’s Tracking Progress is ≥ 50, send 1; when Tracking Progress is < 50, send 0.

© 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
		● bit 7: Tracking status for Own Side Engineer No.2 ● bit 8: Tracking status for Own Side Infantry No.3 ● bit 9: Tracking status for Own Side Infantry Robot No.4 ● bit 10: Own Side Drone special marking status ● bit 11: Own Side Sentry special status ● bit 12: Whether the opponent's Drone is being targeted by own side Radar Laser ● bit 13: Whether the opponent's Drone is currently being countered ● bit 14: Whether own side Drone is being targeted by the opponent's Radar Laser ● bit 15: Whether own side Drone is in a countermeasure state	

```
typedef _packed struct
{
  uint16_t tracking_progress;
}radar_mark_data_t;
```

Table 1-23 0x020D

Byte offset	Size	Description
0	4	● bit 0-10: Except for Remote Exchange, the Projectile Allowance that the Sentry successfully exchanges; it starts at 0 at the beginning of the round, and after the Sentry successfully exchanges a certain Projectile Allowance, this value will change to match the Projectile Allowance exchanged by the Sentry. ● bit 11-14: The number of times the Sentry has successfully performed Remote Exchange for Projectile Allowance. This value starts at 0 at the beginning of the Round, and after the Sentry successfully completes Remote Exchange for Projectile Allowance, it will update to reflect the number of such exchanges performed by the Sentry. ● bit 15-18: Number of times the Sentry successfully performs Remote Exchange for HP. At the start of the Round, this value is 0; after the Sentry completes Remote Exchange for HP, the value updates to reflect the total successful Remote Exchange for HP actions. ● bit 19: Indicates whether the Sentry can confirm free respawn. The value is 1 when free respawn can be confirmed, otherwise it is 0.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
		● bit 20: Indicates whether Sentry can perform an Instant Respawn Exchange at this time. The value is 1 if Instant Respawn Exchange is available, otherwise it is 0. ● bit 21-30: The number of Gold Coins required for Sentry to Exchange for Instant Respawn at the current moment. ● bit 31: Reserved.
4	2	● bit 0: Whether the Sentry is currently out of combat; set to 1 when out of combat, otherwise set to 0. ● bit 1–11: Remaining Exchange count for the team's 17mm Projectile Allowance. ● bit 12-13: Current mode of Sentry; 1 indicates offensive, 2 indicates defensive, and 3 indicates mobile. ● bit 14: Indicates whether the Own Side Power Rune can enter the activating state; 1 means it is currently available. ● bit 15: Whether the Sentry is in a strenthened mode
6	8	● bit 0-7: Remaining duration before the Sentry offensive mode weakens (unit: seconds, same below) ● bit 8-15: Remaining duration before the Sentry defensive mode weakens ● bit 16-23: Remaining duration before the Sentry mobile mode weakens ● bit 24-31: Reserved ● bit 32-39: Remaining duration of Sentry strenthened offensive mode ● bit 40-47: Remaining duration of Sentry strenthened defensive mode ● bit 48-55: Remaining duration of Sentry strenthened mobile mode ● bit 56-63: Reserved

```
typedef _packed struct
{
  uint32_t sentry_info;
  uint16_t sentry_info_2;
  uint64_t sentry_info_3;
}sentry_info_t;
```

© 2026 DJI All Rights Reserved

Table 1-24 0x020E

Byte offset	Size	Description
0	1	• bit 0-1: Indicates whether Radar has triggered double vulnerability; starts at 0. The value represents the number of opportuinty available for Radar to trigger double vulnerability, up to a maximum of 2. • bit 2: Whether the Opponent is currently triggered for double vulnerability. ➢ 0: Opponent has not been triggered for double vulnerability. ➢ 1: Opponent is currently being triggered for double vulnerability. • bit 3-4: Own Side Level (corresponding to Opponent Interference Difficulty Level), starts at 1, with a maximum of 3. • bit 5: Indicates whether the key can currently be modified; 1 means modification is allowed. • bit 6–7: Reserved

```
typedef _packed struct
{
  uint8_t radar_info;
}radar_info_t;
```

Robots interaction data is transmitted via the standard data link, and its data segment includes a unified data segment header structure. The data segment header structure includes the content ID, sender and receiver IDs, and the content data segment. The total length of a robot interaction data packet must not exceed 127 bytes. After subtracting 9 bytes for the frame_header, cmd_id, and frame_tail, as well as 6 bytes for the data segment header structure, the maximum size of the robot interaction content data segment is 112 bytes.

Within every 1000 milliseconds, Hero, Engineer, Infantry, Drone, and Dart can receive up to 3,720 bytes of data, while Radar and Sentry can receive up to 5,120 bytes.

There are multiple content IDs; however, the aggregate uplink update rate for cmd_id is limited to 30 Hz. Please schedule bandwidth usage accordingly.

Table 1-25 0x0301

Byte offset	Size	Description	Remarks
0	2	Sub-content ID	Must be an open sub-content ID.
2	2	Sender ID	Must match Own Side ID; see appendix for ID numbers.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
4	2	Receiver ID	● Own Side communication only. ● Must be an Inter-Robot Communication recipient permitted by the rules. ● If the recipient is the player's client, except for the Radar system, messages can only be sent to the sender's corresponding Player's Client. The Radar system can send to any own side player's client. ● ID numbers are detailed in the appendix.
6	x	Content data segment	Maximum x is 112.

```
typedef _packed struct
{
  uint16_t data_cmd_id;
  uint16_t sender_id;
  uint16_t receiver_id;
  uint8_t user_data[x];
}robot_interaction_data_t;
```

Sub-content ID	Length of the data segment	Instruction
0x0200–0x02FF	x ≤ 112	Communication between robots
0x0100	2	Player's Client deletes a layer
0x0101	15	Player's Client draws a figure
0x0102	30	Player's Client draws two figures
0x0103	75	Player's Client draws five figures
0x0104	105	Player's Client draws seven figures
0x0110	45	Player's Client draws character Figures
0x0120	4	Sentry Self-decision Command
0x0121	8	Radar Self-decision Command

Table 1-26 Sub-content ID: 0x0100

Byte offset	Size	Description	Remarks
0	1	Delete operation	● 0: No operation

© 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
			• 1: Delete layer • 2: Delete all
1	1	Number of layers	Number of layers: 0 to 9

```
typedef _packed struct
{
  uint8_t delete_type;
  uint8_t layer;
}interaction_layer_delete_t;
```

Table 1-27 Sub-content ID: 0x0101

Byte offset	Size	Description	Remarks
0	3	Figure name	Used as an index during operations such as Figure deletion and modification.
3	4	Figure Configuration 1	• bit 0-2: Figure operation ➢ 0: No operation ➢ 1: Add ➢ 2: Edit ➢ 3: Delete • bit 3-5: Figure type ➢ 0: Straight line ➢ 1: Rectangle ➢ 2: Perfect circle ➢ 3: Ellipse ➢ 4: Arc ➢ 5: Floating-point number ➢ 6: Integer ➢ 7: Character • bit 6–9: Number of layers (0–9) • bit 10-13: Color ➢ 0: Red/Blue (Own Side color) ➢ 1: Yellow ➢ 2: Green ➢ 3: Orange ➢ 4: Magenta

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
			➢ 5: Pink ➢ 6: Cyan ➢ 7: Black ➢ 8: White ● bit 14-31: The meaning varies depending on the type of figure being drawn. For details, see "Table 1-27 Instruction for Figure Detail Parameters"
7	4	Figure Configuration 2	● bit 0-9: Line width. It is recommended that the font size to line width ratio be 10:1 ● bit 10–20: Starting point/center x-coordinate ● bit 21-31: Starting point/center y-coordinate
11	4	Figure Configuration 3	Depending on the figure being drawn, the meaning varies. For details, see "Table 1-27 Instruction for Figure Detail Parameters"

```
typedef _packed struct
{
  uint8_t figure_name[3];
  uint32_t operate_type:3;
  uint32_t figure_type:3;
  uint32_t layer:4;
  uint32_t color:4;
  uint32_t details_a:9;
  uint32_t details_b:9;
  uint32_t width:10;
  uint32_t start_x:11;
  uint32_t start_y:11;
  uint32_t details_c:10;
  uint32_t details_d:11;
  uint32_t details_e:11;
}interaction_figure_t;
```

Table 1-28 Instruction for Figure Detail Parameters

Type	details_a	details_b	details_c	details_d	details_e
Straight line	-	-	-	End x coordinate	End y coordinate

© 2026 DJI All Rights Reserved

Type	details_a	details_b	details_c	details_d	details_e
rectangle	-	-	-	Diagonal vertex x coordinate	Diagonal vertex y coordinate
Perfect circle	-	-	Radius	-	-
Ellipse	-	-	-	x semi-axis length	y semi-axis length
Arc	Starting angle	End angle	-	x semi-axis length	y semi-axis length
Floating-point number	Font size	No effect	This value divided by 1,000 is the actual display value.		
Integer	Font size	-	32-bit integer, int32_t		
Character	Font size	Character length	-	-	-

- The angle value means that 0° points to the 12 o'clock direction and is drawn clockwise.
- Screen location: (0, 0) indicates the bottom-left corner, while (1920, 1080) indicates the top-right corner of the screen.
- Floating-point numbers: All integers are 32-bit. For floating-point numbers, the value displayed is the input value divided by 1,000. For example, if you enter 1234 in the bytes corresponding to details_c, details_d, or details_e, the Player's Client will actually display the value as 1.234.
- Even if the transmitted value exceeds the limits of the corresponding data type, the figure may still be displayed, but the display effect is not guaranteed.

Table 1-29 Subcontent ID: 0x0102

Byte offset	Size	Description	Remarks
0	15	Figure 1	Same as the data segment of 0x0101

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
15	15	Figure 2	Same as the data segment of 0x0101

```
typedef _packed struct
{
  interaction_figure_t interaction_figure[2];
}interaction_figure_2_t;
```

Table 1-30 Subcontent ID: 0x0103

Byte offset	Size	Description	Remarks
0	15	Figure 1	Same as the data segment of 0x0101.
15	15	Figure 2	Same as the data segment of 0x0101.
30	15	Figure 3	Same as the data segment of 0x0101.
45	15	Figure 4	Same as the data segment of 0x0101.
60	15	Figure 5	Same as the data segment of 0x0101.

```
typedef _packed struct
{
  interaction_figure_t interaction_figure[5];
}interaction_figure_3_t;
```

Table 1-31 Sub-content ID: 0x0104

Byte offset	Size	Description	Remarks
0	15	Figure 1	Same as the data segment of 0x0101.
15	15	Figure 2	Same as the data segment of 0x0101.
30	15	Figure 3	Same as the data segment of 0x0101.
45	15	Figure 4	Same as the data segment of 0x0101.
60	15	Figure 5	Same as the data segment of 0x0101.
75	15	Figure 6	Same as the data segment of 0x0101.
90	15	Figure 7	Same as the data segment of 0x0101.

```
typedef _packed struct
{
  interaction_figure_t interaction_figure[7];
}interaction_figure_4_t;
```

© 2026 DJI All Rights Reserved

Table 1-32 Sub-content ID: 0x0110

Byte offset	Size	Description	Remarks
0	2	Content ID of the data	0x0110
2	2	Sender's ID	Need to verify the sender's ID for accuracy
4	2	Receiver ID	The receiver's ID must be verified for accuracy; only sending to the Player's Client corresponding to the robot is supported.
6	15	Character configuration	For more details, see the figure data introduction.
21	30	Character	-

```
typedef _packed struct
{
  graphic_data_struct_t grapic_data_struct;
  uint8_t data[30];
}ext_client_custom_character_t;
```

Table 1-33 Sentry Decision Command: 0x0120

Byte offset	Size	Description	Remarks
0	4	Sentry decision-related commands	● bit 0: Whether the Sentry has confirmed respawn ➢ 0 means the Sentry confirms not to respawn, even if the Respawn Timer has finished. ➢ 1 indicates the Sentry confirms revival, and if the Respawn Timer is complete, it will be revived immediately. ● bit 1: Sentry confirms Exchange for Instant Respawn or not ➢ 0 means the Sentry confirms not to exchange for Instant Respawn. ➢ 1 indicates that the Sentry confirmed to Exchange Instant Respawn. If the Sentry meets the requirements for Exchange Instant Respawn at this time, it will immediately spend Gold Coin for Exchange Instant Respawn. ● bit 2-12: The projectile amount value that the Sentry plans to Exchange, which starts at 0. After modifying this value, the Sentry can exchange for the Projectile Allowance at the Restoration Zone. This value must increase monotonically; otherwise, it will be considered invalid.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
			Example: This value can only be 0 at the beginning of the Round. Afterwards, the Sentry can change it from 0 to X, consuming X Gold Coin to successfully exchange for X Projectile Allowance. Afterward, Sentry can change it from X to X+Y, and so on. ● bit 13-16: Number of times the Sentry requests Remote Exchange for Projectile Allowance. The initial value is 0, and updating this value allows the Sentry to request Remote Exchange for Projectile Allowance. This value must increase monotonically, and each change can only increment by 1; otherwise, it will be considered invalid. **Example: This value can only be 0 at the start of the Round. Afterwards, Sentry can change it from 0 to 1, which will consume Gold Coin for a Remote Exchange of Projectile Allowance. Afterward, Sentry can change the value from 1 to 2, and so on.** ● bit 17-20: Number of times Sentry requests remote Exchange of health points. The initial value is 0, and changing this value allows the Sentry to request remote exchange for HP. This value must increase monotonically, and each change can only increment by 1; otherwise, it will be considered invalid. **Example: This value can only be 0 at the start of the round. Afterwards, Sentry can change it from 0 to 1, which consumes Gold Coin for Remote Exchange of HP. Afterward, Sentry can change the value from 1 to 2, and so on.** When Sentry sends this sub-command, the Server processes these Commands sequentially from lower to higher bits, until all are successful or cannot be processed further. **Example: If the team's Gold Coin count is 0 and the Sentry is defeated, the value for "Confirm respawn" is 1, the value for "Conform Instant Respawn Exchange" is 1, and the value for "Confirm Exchange for Projectile Allowance" is 100. (Assuming the Sentry has not previously exchanged Projectile Allowance) Since the team's Gold Coin balance is insufficient for the Sentry to exchange for Instant Respawn, the Server will**

© 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
			disregard subsequent commands and wait for the next set of commands sent by the Sentry. ● bit 21-23: Sentry modifies its current mode command, with 1 for offensive, 2 for defensive, and 3 for mobile. The default is 3; changing this value updates the Sentry's mode; 4 for strengthened offensive; 5 for strengthened defensive; 6 for strengthened mobile. The default is 3; modifying this value changes the Sentry mode. ● bit 24: Whether the Sentry confirms to set the Power Rune to activating status; 1 means yes. Default value is 0. ● bit 25-31: Reserved.

```
typedef _packed struct
{
  uint32_t sentry_cmd;
}sentry_cmd_t;
```

Table 1-34 Radar Decision Command: 0x0121

Byte offset	Size	Description	Remarks
0	1	Whether the Radar confirms to trigger double vulnerability	At the start of the round, the value is set to 0. You can request to trigger double vulnerability by modifying this value. If Radar has an opportuinty to trigger double vulnerability at this time, it will be triggered. This value must increase monotonically, and each change can only increment by 1; otherwise, it will be considered invalid. **Example: This value must be 0 at the beginning of the Round. Afterwards, Radar can change it from 0 to 1. If Radar has a chance to trigger double vulnerability, double vulnerability will be triggered. Afterward, Radar can change the value from 1 to 2, and continue in this manner.** If Radar requests double vulnerability while double vulnerability is already active, the second double vulnerability will become active once the first one ends.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
1	7	Command for key update or verification	Each byte is encoded as an ASCII letter or number. Starts with a random value in each Round. byte1 indicates the Command type, while byte2-7 represent the key value. When the value of byte1 is 1, modifying this value will update Own Side's encrypted key; when the value of byte1 is 2, modifying this value will transmit the Opponent's key, which Radar has decoded, to the Server to verify if it has been decoded correctly. Note: ● You can only change the key at the beginning of the round, or when your Opponent successfully cracks it and the encryption Level (Own Side interference difficulty) increases. Any other attempts to modify the key will not be valid. (Even if not actively modified, the key will be reset to a new random value) ● When byte1 is set to 2, any further updates within 10 seconds after the password is verified and updated will not be effective.

```
typedef _packed struct
{
  uint8_t radar_cmd;
  uint8_t password_cmd;
  uint8_t password_1;
  uint8_t password_2;
  uint8_t password_3;
  uint8_t password_4;
  uint8_t password_5;
  uint8_t password_6;
}radar_cmd_t;
```

1.3 Instructions for Mini-map Interaction Data

1.3.1 Sending Data via Player's Client

- The Gimbal Operator can use the Player's Client main map to send predefined data to the robot.

Command code is 0x0303, transmitted when triggered, with a minimum interval of 0.5 seconds

© 2026 DJI All Rights Reserved

between each transmission.

Sending method 1:

① Click on the Own Side robot's icon.

② (Optional) Press a key on the keyboard, or click the Opponent robot's avatar;

③ Click anywhere on the Mini-map. This method sends map coordinate data to the selected Own Side robot; if you click on an Opponent Robot avatar, the target robot's ID will be sent instead of the coordinate data.

Sending Method 2:

① (Optional) Press a keyboard key or click the Opponent robot's avatar;

② Click anywhere on the Mini-map. This method sends map coordinate data to all robots on Own Side. If you click on the Opponent robot's avatar, the target robot ID will replace the coordinate data

- Operators corresponding to robots in semi-automatic control mode can send preset data to Robots via the Player's Client main map.

The command code is 0x0303. It is sent when triggered, and the interval between two transmissions must not be less than 3 seconds.

Sending Method:

① (Optional) Press a keyboard key or click the Opponent robot's avatar.

② Click anywhere on the Mini-map. This method sends map coordinate data to the robot associated with the Operator. If the Opponent's robot avatar is clicked, the target Robot ID will replace the coordinate data.

A single robot in semi-automatic control mode can receive information sent by the Gimbal Operator as well as by the corresponding Operator. The sources of the two types of information are distinguished in the "Information Source" column in the table below.



To reduce the impact of occasional instability in robot serial port receiver devices on communication, the sending mechanism for the 0x0303 protocol has a special arrangement: after the Player's Client triggers a single transmission, the Server will send the same message to the robots an additional four times at 100ms intervals, for a total of five transmissions. Afterwards, until the next time the Player's Client triggers a transmission, the Server will continue to send the most recent packet at a fixed frequency of 1Hz. The timing for continuous sending upon trigger and the timing for fixed 1Hz frequency transmission operate independently. Teams need to pay attention to how repeatedly received duplicate

 © 2026 DJI All Rights Reserved

protocol packets are handled.

Table 1-35 Command ID: 0x0303

Byte offset	Size	Description	Remarks
0	4	X-axis coordinate of the target location, in meters	When sending the Robot ID, this value is 0.
4	4	y-axis coordinate of the target location, in meters	When sending the Robot ID, this value is 0.
8	1	General key value for the keyboard key pressed by the Gimbal Operator	If no key is pressed, the value is 0.
9	1	Opponent Robot ID	When sending coordinate data, this value is 0.
10	2	Information source ID	ID of the information source, with detailed ID mapping provided in the appendix.

```
typedef _packed struct
{
  float opponent_position_x;
  float opponent_position_y;
  uint8_t cmd_keyboard;
  uint8_t opponent_robot_id;
  uint16_t source_id;
}map_command_t;
```

1.3.2 Receiving Data via Player's Client

The Mini-map on the Player's Client can receive data from robots.

Radar can transmit the coordinates of both side robots to all Player's Clients on Own Side via a standard data link, and these locations will be displayed on the Mini-map of Own Side Player's Clients.

Table 1-36 Command ID: 0x0305

Byte offset	Size	Description	Remarks
0	2	Opponent Hero X-coordinate, unit: cm	
2	2	Opponent Hero Y-coordinate, unit: cm	
4	2	Opponent Engineer X-coordinate, unit: cm	

© 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
6	2	Opponent Engineer Y-coordinate, unit: cm	If the x or y values exceed the boundaries, the point will appear at the corresponding edge. If both x and y are zero, it is considered that the robot's location data has not been sent.
8	2	Opponent No.3 Infantry X-coordinate, unit: cm	
10	2	Opponent No.3 Infantry Y-coordinate, unit: cm	
12	2	Opponent No.4 Infantry X-coordinate, unit: cm	
14	2	Opponent No.4 Infantry Y-coordinate, unit: cm	
16	2	Opponent No.6 Drone X-coordinate, unit: cm	
18	2	Opponent No.6 Drone Y-coordinate, unit: cm	
20	2	Opponent Sentry X-coordinate, unit: cm	
22	2	Opponent Sentry Y-coordinate, unit: cm	
24	2	Own side Hero X-coordinate, unit: cm	
26	2	Own side Hero Y-coordinate, unit: cm	
28	2	Own side Engineer X-coordinate, unit: cm	
30	2	Own side Engineer Y-coordinate, unit: cm	
32	2	Own side No.3 Infantry X-coordinate, unit: cm	
34	2	Own side No.3 Infantry Y-coordinate, unit: cm	
36	2	Own side No.4 Infantry X-coordinate, unit: cm	
38	2	Own side No.4 Infantry Y-coordinate, unit: cm	
40	2	Own side No.6 Drone X-coordinate, unit: cm	

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
42	2	Own side No.6 Drone Y-coordinate, unit: cm	
44	2	Own side Sentry X-coordinate, unit: cm	
46	2	Own side Sentry Y-coordinate, unit: cm	

```
typedef _packed struct
{
  uint16_t opponent_hero_position_x;
  uint16_t opponent_hero_position_y;
  uint16_t opponent_engineer_position_x;
  uint16_t opponent_engineer_position_y;
  uint16_t opponent_infantry_3_position_x;
  uint16_t opponent_infantry_3_position_y;
  uint16_t opponent_infantry_4_position_x;
  uint16_t opponent_infantry_4_position_y;
  uint16_t opponent_aerial_position_x;
  uint16_t opponent_aerial_position_y;
  uint16_t opponent_sentry_position_x;
  uint16_t opponent_sentry_position_y;
  uint16_t ally_hero_position_x;
  uint16_t ally_hero_position_y;
  uint16_t ally_engineer_position_x;
  uint16_t ally_engineer_position_y;
  uint16_t ally_infantry_3_position_x;
  uint16_t ally_infantry_3_position_y;
  uint16_t ally_infantry_4_position_x;
  uint16_t ally_infantry_4_position_y;
  uint16_t ally_aerial_position_x;
  uint16_t ally_aerial_position_y;
  uint16_t ally_sentry_position_x;
  uint16_t ally_sentry_position_y;
}map_robot_data_t;
```

Sentry or semi-automated robots can send path coordinate data via a standard data link to the corresponding Player's Client, and the path will be displayed on the Mini-map.

© 2026 DJI All Rights Reserved

Table 1-37 Command Code ID: 0x0307

Byte offset	Size	Description	Remarks
0	1	1: Move to the designated location and attack 2: Move to the designated location and defend 3: Move to the designated location	-
1	2	Starting x-axis coordinate of the path, uint: dm	The bottom-left corner of the mini-map is the origin of the coordinate system, with the positive X-axis extending to the right horizontally and the positive Y-axis extending upward vertically. The display location will be scaled proportionally according to the match area size and mini-map size, and any location outside the boundaries will be shown at the edge.
3	2	Starting y-axis coordinate of the path, uint: dm	
5	49	Array of X-axis increments for path points, uint: dm	Incremental values are calculated relative to the previous position, with 49 new positions in total. Each position is determined by corresponding increments along the X and Y axes.
54	49	Array of y-axis increments for path points, uint: dm	
103	2	Sender ID	Must match Own Side ID; see appendix for ID codes.

```
typedef _packed struct
{
  uint8_t intention;
  uint16_t start_position_x;
  uint16_t start_position_y;
  int8_t delta_x[49];
  int8_t delta_y[49];
  uint16_t sender_id;
}map_data_t;
```

Own Side robots can use a standard data link to send customized messages to any Own Side Player's Client, and these messages will be displayed at a specific location on the Own Side Player's Client.

 © 2026 DJI All Rights Reserved

Table 1-38 Command Code ID: 0x0308

Byte offset	Size	Description	Remarks
0	2	Sender ID	Need to verify the sender's ID for accuracy.
2	2	Receiver ID	The receiver's ID must be validated for accuracy; data can only be sent to Own Side Player's Client.
4	30	Character	Encoded in UTF-16 format for transmission, allowing Chinese characters to be displayed. When encoding and sending data, please pay attention to the endianness issue.

```
typedef _packed struct
{
  uint16_t sender_id;
  uint16_t receiver_id;
  uint8_t user_data[30];
}custom_info_t;
```

1.4 Instructions for VTM Link Data

1.4.1 Instructions for Custom Controller and Robots Interaction Data

Operators can use a custom controller to send data to the corresponding robots via the VTM Link.

Table 1-39 Command Code ID: 0x0302

Byte offset	Size	Description
0	30	Custom Data

```
typedef _packed struct
{
  uint8_t data[x];
}custom_robot_data_t;
```

Robots can transmit data via VTM link to the custom controller connected to the corresponding Player's Client.

Table 1-40 Command ID: 0x0309

Byte offset	Size	Description
0	30	Custom Data

```
typedef _packed struct
{
```

© 2026 DJI All Rights Reserved

```
  uint8_t data[x];
}robot_custom_data_t;
```

1.4.2 Instruction for Custom Client and Robots Interaction Data

Robots can use the VTM Link to send custom information to the Custom Client, and receive custom Commands from the Custom Client.

Note: Due to the large volume of data, this protocol does not use a retransmission Mechanism.

Table 1-41 Command ID: 0x0310

Byte offset	Size	Instruction
0	300	Custom data

```
typedef _packed struct
{
  uint8_t data[x];
}robot_custom_ data_2_t;
```

Table 1-42 Command ID: 0x0311

Byte offset	Size	Instruction
0	30	Custom Client sends custom Commands to Robots

```
typedef _packed struct
{
  uint8_t data[x];
}robot_custom_ data_3_t;
```

1.5 Instructions for Non-data Link Data

The Operator can use a custom controller to simulate keyboard and mouse operations on the Player's Client.

Table 1-43 Command Code ID: 0x0306

Byte offset	Size	Description	Remarks
0	2	Keyboard key values: ● bit 0-7: Key value of Button 1 ● bit 8-15: Key 2 value	● Only responds to keys enabled on the Player's Client.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description	Remarks
			• Use general key values, supports 2-key rollover, changing the key value order will not affect the pressed state, and if no new key information is received, the pressed state from the previous frame will be maintained.
2	2	• bit 0-11: Mouse x-axis pixel location • bit 12-15: Mouse left button status	• Location information uses absolute pixel values (the competition client operates at a resolution of 1920×1080, with the top-left corner of the screen as (0, 0)). • When the mouse button status is 1, it indicates the button is pressed; any other value means it is not pressed. This information is only updated when the mouse icon appears. If there is no new mouse data, the Player's Client will retain the mouse information from the previous frame. Once the mouse icon disappears, this data will no longer be retained.
4	2	• bit 0-11: Mouse y-axis pixel location • bit 12-15: Right mouse button status	
6	2	Reserved	-

	To perform a mouse move-and-click action, first send a data frame indicating the mouse is not pressed at the specified location, then send a data frame indicating the mouse is pressed while holding that location, and finally send a data frame indicating the mouse is not pressed while still holding that location.

```
typedef _packed struct
{
  uint16_t key_value;
  uint16_t x_location:12;
  uint16_t mouse_left:4;
  uint16_t y_location:12;
  uint16_t mouse_right:4;
  uint16_t reserved;
}custom_client_data_t;
```

© 2026 DJI All Rights Reserved

1.6 Instructions for Radar Wireless Link

Table 1-44 Command Code ID: 0x0A01

Byte offset	Size	Description
0	2	Opponent Hero position X-coordinate (Uint: cm)
2	2	Opponent Hero position Y-coordinate (Uint: cm)
4	2	Opponent Engineer position X-coordinate (Uint: cm)
6	2	Opponent Engineer position Y-coordinate (Uint: cm)
8	2	Opponent Infantry 3 position X-coordinate (Uint: cm)
10	2	Opponent Infantry 3 position Y-coordinate (Uint: cm)
12	2	Opponent Infantry 4 position X-coordinate (Uint: cm)
14	2	Opponent Infantry 4 position Y-coordinate (Uint: cm)
16	2	Opponent Drone position X-coordinate (Uint: cm)
18	2	Opponent Drone position Y-coordinate (Uint: cm)
20	2	Opponent Sentry position X-coordinate (Uint: cm)
22	2	Opponent Sentry position Y-coordinate (Uint: cm)

Table 1-45 Command Code ID: 0x0A02

Byte offset	Size	Description
0	2	Opponent Hero (No.1) HP If the robot is not fielded or has been disqualified, the HP is 0. This applies to the fields below as well.
2	2	Opponent Engineer (No.2) HP
4	2	Opponent Infantry (No.3) HP
6	2	Opponent Infantry (No.4) HP
8	2	Reserved.
10	2	Opponent Sentry (ID 7) HP

Table 1-46 Command Code ID: 0x0A03

Byte offset	Size	Description
0	2	Opponent Hero (No.1) projectile allowance.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
2	2	Opponent Infantry (No.3) projectile allowance (Includes the reserve projectile allowance provided by the Fortress; same applies below).
4	2	Opponent Infantry (No.4) projectile allowance
6	2	Opponent Drone (No.6) projectile allowance
8	2	Opponent Sentry (No.7) projectile allowance
0	2	Opponent Hero(No.1) projectile allowance

Table 1-47 Command Code ID: 0x0A04

Byte offset	Size	Description
0	2	Opponent remaining gold coins
2	2	Opponent total accumulated gold coins
4	4	● bit 0: Opponent Supply Zone occupation status. ● bit 1-2: Opponent Central Elevated Ground occupation status. 1: Detection of the opponent robot's RFID Interaction Module; 2: occupied by own side ● bit 3: Opponent Trapezoidal Elevated Ground occupation status. 1: occupied ● bit 4-5: Opponent Fortress Buff Point occupation status. 0: unoccupied; 1: Detection of the opponent robot's RFID Interaction Module; 2: occupied by own side; 3: occupied by both ● bit 6-7: Opponent Outpost Buff Point occupation status. 0: unoccupied; 1: Detection of the opponent robot's RFID Interaction Module; 2: occupied by own side ● bit 8: Opponent Base Buff Point occupation status. 1: occupied ● bit 9: Status of the RFID interaction card at the center of the Tunnel (Terrain Crossing Buff Point) before the slope on the opponent's side. 1: Detection of the opponent robot's RFID Interaction Module ● bit 10: Status of the RFID interaction card at the center of the Tunnel after the slope on the opponent's side. 1: Detection of the opponent robot's RFID Interaction Module ● bit 11: Status of the RFID interaction card at the center of the Tunnel before the slope on the own side. 1: Detection of the opponent robot's RFID Interaction Module ● bit 12: Status of the RFID interaction card at the center of the Tunnel after the slope on the own side. 1: Detection of the opponent robot's RFID Interaction Module

© 2026 DJI All Rights Reserved

Byte offset	Size	Description
		● bit 13: Status of the RFID interaction card on top of the opponent's Elevated Ground (Terrain Crossing Buff Point). 1: Detection of the opponent robot's RFID Interaction Module) ● bit 14: Status of the RFID interaction card at the rear of the opponent's Launch Ramp (Terrain Crossing Buff Point). 1: Detection of the opponent robot's RFID Interaction Module ● bit 15: Status of the RFID interaction card on top of the opponent's Road (Terrain Crossing Buff Point). 1: Detection of the opponent robot's RFID Interaction Module

Table 1-48 Command Code ID: 0x0A05

Byte offset	Size	Description
0	1	Opponent Hero HP Recovery Buff (Percentage, value of 10 indicates 10% of max HP recovery per second, same below).
1	2	Opponent Hero Cooling Rate Buff (Value, value of x indicates cooling rate increases by x/s, same below).
3	1	Opponent Hero Defense Buff (Percentage, value of 50 indicates 50% defense buff, same below).
4	1	Opponent Hero Negative Defense Buff (Percentage, value of 30 indicates -30% defense debuff, same below).
5	2	Opponent Hero Attack Buff (Percentage, value of 50 indicates 50% attack buff, same below).
7	1	Opponent Engineer HP Recovery Buff.
8	2	Opponent Engineer Cooling Rate Buff.
10	1	Opponent Engineer Defense Buff.
11	1	Opponent Engineer Negative Defense Buff.
12	2	Opponent Engineer Attack Buff.
14	1	Opponent Infantry (No. 3) HP Recovery Buff.
15	2	Opponent Infantry (No. 3) Cooling Rate Buff.
17	1	Opponent Infantry (No. 3) Defense Buff.
18	1	Opponent Infantry (No. 3) Negative Defense Buff.
19	2	Opponent Infantry (No. 3) Attack Buff.
21	1	Opponent Infantry (No. 4) HP Recovery Buff.

 © 2026 DJI All Rights Reserved

Byte offset	Size	Description
22	2	Opponent Infantry (No. 4) Cooling Rate Buff.
24	1	Opponent Infantry (No. 4) Defense Buff.
25	1	Opponent Infantry (No. 4) Negative Defense Buff.
26	2	Opponent Infantry (No. 4) Attack Buff.
28	1	Opponent Sentry HP Recovery Buff.
29	2	Opponent Sentry Cooling Rate Buff.
31	1	Opponent Sentry Defense Buff.
32	1	Opponent Sentry Negative Defense Buff.
33	2	Opponent Sentry Attack Buff.
35	1	Opponent Sentry Current Posture (1 for offensive, 2 for defensive , 3 for mobile, 4 for strengthened offensive, 5 for strengthened defensive , 6 for strengthened mobile).
36	1	Opponent Hero main status (0 for alive, 1 for defeated, 2 for invincible but not weakened, 3 for invincible and weakened. Offline will not affect the status judgment here, the same below).
37	1	Opponent Engineer main status
38	1	Opponent No.3 Infantry main status
39	1	Opponent No.4 Infantry main status
40	1	Opponent Sentry main status

Table 1-49 Command Code ID: 0x0A06

Byte offset	Size	Description
0	6	Each byte is an ASCII-encoded letter or digit.

© 2026 DJI All Rights Reserved

2 Custom Client Protocol

The data stream format for the custom client uses Protobuf v3 (Protocol Buffers), supporting version 3.15 or above, with MQTT as the message publishing and subscription protocol. MQTT transmits the Protobuf-serialized binary stream directly. The topic corresponds to the command name. The server IP address is set to a fixed IP: 192.168.12.1, and the port is 3333. The custom client IP address is 192.168.12.2.

The communication process is as follows:

1. Write proto files based on the Protobuf message format in the documentation, and use protoc to generate class code in various languages.
2. On the publisher side, serialize the object into binary.
3. Send the binary data to the corresponding topic via MQTT publish.
4. On the subscriber side, receive messages and deserialize them using Protobuf.

Sample code is provided in “Appendix III: Sample Communication Code for Custom Client”. In MyMqttClient, set the clientID to the ID number corresponding to the connected Robots.

In addition, a custom client can access the video transmission stream data by listening to port 3334 via UDP. The encoding format is HEVC, and the first 8 bytes of the stream data in each UDP packet are fixed as follows:

Frame number (incremental): 2 byte

Fragment index within current frame: 2 byte

Total bytes in current frame: 4 byte

2.1 Command Overview

Table 2-1 Command overview

Command (message) name	Command purpose	Sender/ Receiver	Maximum qos Level	Frequency
KeyboardMouseControl	Transmit keyboard and mouse input	Custom Client → VTM Link → Robots	1	75Hz

 © 2026 DJI All Rights Reserved

Command (message) name	Command purpose	Sender/ Receiver	Maximum qos Level	Frequency
CustomControl	Up to 30 bytes of custom data	Custom Client → VTM Link → Robots	1	75Hz
GameStatus	Synchronize the global round status information	Server → Custom Client	1	5Hz
GlobalUnitStatus	Sync the status of Base, Outpost, and all robots	Server → Custom Client	1	1Hz
GlobalLogisticsStatus	Synchronize global logistics information	Server → Custom Client	1	1Hz
GlobalSpecialMechanism	Synchronize active global special mechanism	Server → Custom Client	1	1Hz
Event	Global round event notification	Server → Custom Client	1	Sending upon trigger
RobotInjuryStat	Cumulative damage during one alive period	Server → Custom Client	1	1Hz
RobotRespawnStatus	Robots respawn status synchronization	Server → Custom Client	1	1Hz
RobotStaticStatus	Robots' fixed attributes and configurations	Server → Custom Client	1	1Hz
RobotDynamicStatus	Real-time robot data	Server → Custom Client	1	10Hz

© 2026 DJI All Rights Reserved

Command (message) name	Command purpose	Sender/ Receiver	Maximum qos Level	Frequency
RobotModuleStatus	Operating status of each module of the robots	Server → Custom Client	1	1Hz
RobotPosition	Robot spatial coordinates and orientation	Server → Custom Client	1	1Hz
Buff	Buff effect information	Server → Custom Client	1	Triggered to send upon gaining a buff, then sent at a fixed frequency of 1Hz until the buff ends
PenaltyInfo	Penalty information synchronization	Server → Custom Client	1	Sending upon trigger
RobotPathPlanInfo	Sentinel trajectory planning information	Server → Custom Client	1	Sending upon trigger
MapClickInfo	Map click marker information synchronization	Server → Custom Client	1	The frequency limit for Trigger-based transmissions is consistent with “1.3.1 Sending Data via Player’s Client”

 © 2026 DJI All Rights Reserved

Command (message) name	Command purpose	Sender/ Receiver	Maximum qos Level	Frequency
MapClickCmd	Map click marker command	Custom Client→ Server	1	The frequency limit for Trigger-based transmissions is consistent with “1.3.1 Sending Data via Player’s Client”
RadarInfoToClient	Location information of robots sent by Radar	Server → Custom Client	1	Sending upon trigger
CustomByteBlock	Custom data stream uploaded by robots (corresponding to 0x0310 on the Onboard Modules)	Robots → VTM Link → Custom Client	1	50Hz
AssemblyCommand	Engineer Assembly Command	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
TechCoreMotionStateSync	Tech Core motion state sync	Server → Custom Client	1	1Hz
RobotPerformanceSelectionCommand	Ground Robot Performance System or Control Mode Selection	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz

© 2026 DJI All Rights Reserved

Command (message) name	Command purpose	Sender/ Receiver	Maximum qos Level	Frequency
RobotPerformanceSelectionSync	Infantry/Hero Performance System status sync	Server → Custom Client	1	1Hz
CommonCommand	Various common instructions for robots	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
HeroDeployModeEventCommand	Commands related to Hero Deploy Mode	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
DeployModeStatusSync	Deploy mode status synchronization	Server → Custom Client	1	1Hz
RuneActivateCommand	Power Rune activation command	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
RuneStatusSync	Power Rune status sync	Server → Custom Client	1	1Hz
SentryStatusSync	Sentry pose status sync	Server → Custom Client	1	1Hz

 © 2026 DJI All Rights Reserved

Command (message) name	Command purpose	Sender/ Receiver	Maximum qos Level	Frequency
DartCommand	Dart control command	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
DartSelectTargetStatusSync	Dart selection status synchronization	Server → Custom Client	1	1Hz
SentryCtrlCommand	Sentry control command request	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
SentryCtrlResult	Sentry control command result feedback	Server → Custom Client	1	Triggered sending, with a maximum sending frequency of 10Hz
AirSupportCommand	Air support command	Custom Client→ Server	1	Triggered sending, with a maximum sending frequency of 10Hz
AirSupportStatusSync	Air support status feedback	Server → Custom Client	1	1Hz

2.2 Detailed protocol specification

2.2.1 KeyboardMouseControl

Purpose: to transmit mouse and keyboard input.

© 2026 DJI All Rights Reserved

This data is sent to Robots through the Video Transmitter Module (Receiver). During actual matches, the Operator must select in advance whether keyboard and mouse Control Commands come from the official Player's Client or a custom Client, and can generate keyboard and mouse Commands for the Robots from only one source at a time.

Data ID	Data type	Data purpose
1	int32	Mouse x-axis movement speed, with negative values indicating movement to the left
2	int32	Mouse y-axis movement speed, with negative values indicating downward movement
3	int32	Mouse scroll wheel speed; negative values indicate scrolling backward
4	bool	Whether the left mouse button is pressed (false = released, true = pressed)
5	bool	Is the right mouse button pressed (false = released, true = pressed)
6	uint32	Keyboard key bitmask
7	bool	Is the middle button pressed (false = released, true = pressed)

Keyboard key bitmask: Note: Although the data type is uint32, in practice you only need to send data for bits 0 through 15.

Correspondence:

Each bit corresponds to one key: 0 means not pressed, and 1 means pressed.

- bit 0：W key
- bit 1：S key
- bit 2：A key
- bit 3：D key
- bit 4：Shift key
- bit 5：Ctrl key
- bit 6：Q key

 © 2026 DJI All Rights Reserved

- bit 7：E key
- bit 8：R key
- bit 9：F key
- bit 10：G key
- bit 11：Z key
- bit 12：X key
- bit 13：C key
- bit 14：V key
- bit 15：B key

```
message RemoteControl {
  optional int32 mouse_x = 1;
  optional int32 mouse_y = 2;
  optional int32 mouse_z = 3;
  optional bool left_button_down = 4;
  optional bool right_button_down = 5;
  optional uint32 keyboard_value = 6;
  optional bool mid_button_down = 7;
}
```

2.2.2 CustomControl

Purpose: Up to 30 bytes of custom data.

Instruction: This information is sent to Robots over the VTM Link using Command code 0x0311.

Data ID	Data type	Data purpose
1	bytes	Maximum 30-byte custom data

```
message CustomControl {
  optional bytes data=1;
}
```

© 2026 DJI All Rights Reserved

2.2.3 GameStatus

Purpose: Synchronize the overall competition status information.

Data ID	Data type	Data purpose
1	uint32	Current round number (starting from 1)
2	uint32	Total number of rounds
3	uint32	Red Team score
4	uint32	Blue Team score
5	uint32	Current phase
6	int32	Remaining time in current phase (seconds)
7	int32	Elapsed time in current stage (seconds)
8	bool	Is the match paused
9	uint32	Winner of the match. 0 for draw, 1 for red victory, 2 for blue victory. This value is only meaningful when the 'current phase' is 'match settlement', otherwise, send 255
10	uint32	Reason for match end. This value is only meaningful when the 'current phase' is 'match settlement', otherwise, send 255

Enumeration values for current_stage:

- 0: Match Not Started
- 1: Preparation Period
- 2: 15-Second Referee System Initialization Period
- 3: 5-Second Countdown
- 4: In Match
- 5: Match Settling

Enumeration values for end_reason:

- 1: Base destroyed

 © 2026 DJI All Rights Reserved

- 2: Base Remaining HP
- 3: Outpost Remaining HP
- 4: Whether the Outpost has been destroyed
- 5: Total team Attack Damage
- 6: Total remaining HP of all team Robots
- 7: Draw
- 8: Abnormal termination
- 9: The referee manually determines the winner

```
message GameStatus {
  optional uint32 current_round = 1;
  optional uint32 total_rounds = 2;
  optional uint32 red_score = 3;
  optional uint32 blue_score = 4;
  optional uint32 current_stage = 5;
  optional int32 stage_countdown_sec = 6;
  optional int32 stage_elapsed_sec = 7;
  optional bool is_paused = 8;
  optional uint32 game_result = 9;
  optional uint32 end_reason =10;
}
```

2.2.4 GlobalUnitStatus

Purpose: synchronize the Base, Outpost, and all Robots states.

Data ID	Data type	Data purpose
1	uint32	Current HP of Own Side Base (0 = destroyed)
2	uint32	Own Side Base status
3	uint32	Current shield value of Own Side Base (0 = no shield)
4	uint32	Own Side Outpost current HP (0 = destroyed)
5	uint32	Own Side Outpost status

© 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
6	uint32	Opponent Base current HP (0 = destroyed)
7	uint32	Opponent Base status
8	uint32	Opponent Base current shield value (0 = no shield)
9	uint32	Opponent Outpost current HP (0 = destroyed)
10	uint32	Opponent Outpost status
11	repeated uint32	Health of all robots (sent in the order of Own Side: No. 1, 2, 3, 4, 7, and Opponent: No. 1, 2, 3, 4, 7)
12	repeated int32	Remaining total ammuintion fired by Own Side Robots
13	uint32	Own Side total accumulated Damage
14	uint32	Opponent total accumulated Damage

base_status enumeration values:

- 0: Invincible
- 1: Own Side is no longer Invincible, armor not deployed
- 2: Invincible removed, armor deployed

outpost_status enumeration values:

- 0: Invincible
- 1: Alive, Invincible removed, central armor rotating
- 2: Alive, Invincible removed, central armor stopped rotating
- 3: Destroyed, cannot be rebuilt
- 4: Destroyed, can be rebuilt
- 5: Destroyed, rebuilding underway

```
message GlobalUnitStatus {
  optional uint32 base_health = 1;
  optional uint32 base_status = 2;
  optional uint32 base_shield = 3;
```

 © 2026 DJI All Rights Reserved

```
  optional uint32 outpost_health = 4;
  optional uint32 outpost_status = 5;
  optional uint32 enemy_base_health = 6;
  optional uint32 enemy_base_status = 7;
  optional uint32 enemy_base_shield=8;
  optional uint32 enemy_outpost_health = 9;
  optional uint32 enemy_outpost_status = 10;
  repeated uint32 robot_health = 11;
  repeated int32 robot_bullets = 12;
  optional uint32 total_damage_ally = 13;
  optional uint32 total_damage_enemy = 14;
}
```

2.2.5 GlobalLogisticsStatus

Purpose: Synchronize global logistics information.

Data ID	Data type	Data purpose
1	uint32	Own Side current economy
2	uint64	Own Side total accumulated economy
3	uint32	Own Side Tech Core Level (The highest assembly difficulty level previously accomplished by the Engineer)
4	uint32	Own Side encryption level (Radar signal information mechanism)

```
message GlobalLogisticsStatus {
  optional uint32 remaining_economy = 1;
  optional uint64 total_economy_obtained = 2;
  optional uint32 own_side_level = 3;
  optional uint32 encryption_level = 4;
}
```

2.2.6 GlobalSpecialMechanism

Purpose: Synchronize globally active special mechanisms.

Data ID	Data type	Data purpose
1	repeated uint32	List of active mechanism IDs

© 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
2	repeated int32	Corresponding time parameters (seconds)

Enumeration of mechanism_id values:

- 1: Timer for Own Side Fortress being occupied by Opponent.
- 2: Timer for Opponent Fortress being occupied by Own Side.

```
message GlobalSpecialMechanism {
  optional repeated uint32 mechanism_id = 1;
  optional repeated int32 mechanism_time_sec = 2;
}
```

2.2.7 Event

Purpose: Global event notification.

Data ID	Data type	Data purpose
1	int32	Event ID
2	string	Event parameters (meaning varies depending on event_id)

Enumeration of event_id values:

- 1: Elimination event (parameter 1 is the defeated unit, parameter 2 is the unit that defeats it; for example, "1,101" means Blue Team Hero destroys Red Team Hero).
- 2: Outpost destruction event (parameter value is the destroyed target ID, with Red Team Outpost as 11, and the Blue Team outpost as 111).
- 3: The number of the Large Power Rune Light Arms successfully activated and the average number of Rings (this event is triggered only when the Large Power Rune is activated, and the parameter value is the number of successfully activated Arms and the average number of Rings (floating point number), for example, "10,9.6" represents ten Light Arms activated, with an average of 9.6 Rings).
- 4: Power Rune enters activated state (activation type 1: Small Power Rune, activation type 2: Large Power Rune).
- 5: Own Side Hero inflicts sniper damage (parameter is the total sniper damage inflicted).
- 6: Opponent Hero dealt sniper damage (parameter value is the total sniper damage inflicted).
- 7: Opponent calls for air support (no parameter value).

 © 2026 DJI All Rights Reserved

- 8: Own Side Hero inflicts sniper damage (parameter is the total sniper damage inflicted).
- 9: Dart hit (the first parameter indicates which side scored the hit: 1 for the Red Team, 2 for the Blue Team; the second parameter indicates what was hit: 1 for the Outpost, 2 for a fixed point in the Base, 3 for a random fixed point in the Base, 4 for a random moving point in the Base, 5 for an end-section moving point in the Base).
- 10: Opponent dart gates open (no parameters).
- 11: Base is under attack (no parameter value; each trigger has a built-in 5 s cooldown).
- 12: Opponent Outpost has stopped rotating (no parameters).
- 13: Opponent Base Protective Armor deployed (no parameters).
- 14: During assembly, the other party requests level-4 assembly, triggering entry into the forced exit buffer period (no parameters).
- 15: Assembly result event (0 success, 1 removed, 2 timeout, 3 stayed out of the assembly area for too long, 4 engineer eliminated, 5 level 4 difficulty failed to meet the cooperation time limit, 6 exited manually, 7 assembly completed but no Energy Unit detected at settlement, 8 buffer period expired and the assembly process was forcibly ended).

```
message Event {
  optional int32 event_id = 1;
  optional string param = 2;
}
```

2.2.8 RobotInjuryStat

Purpose: Cumulative damage statistics for robots during a single alive period.

Data ID	Data type	Data purpose
1	uint32	Total damage accumulated during this alive period
2	uint32	Collision damage
3	uint32	17mm Projectile damage
4	uint32	42mm projectile damage
5	uint32	Dart splash damage
6	uint32	Module Offline HP Loss

© 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
7	uint32	Irregular Offline HP Loss
8	uint32	Penalty HP Loss
9	uint32	Server forces HP Loss by marking as defeated.
10	uint32	Destroyer ID (if no destroyer is detected, the value is 0)

```
message RobotInjuryStat {
  optional uint32 total_damage = 1;
  optional uint32 small projectile damage = 3;
  optional uint32 large projectile damage = 4;
  optional uint32 dart splash damage = 5;
  optional uint32 module_offline_damage = 6;
  optional uint32 offline_damage = 7;
  optional uint32 penalty_damage = 8;
  optional uint32 server_kill_damage = 9;
  optional uint32 killer_id = 10;
}
```

2.2.9 RobotRespawnStatus

Purpose: Robots Revive status synchronization.

Data ID	Data type	Data purpose
1	bool	Is the robot in pending respawn state
2	uint32	Total Respawn Timer required for respawn
3	uint32	Current Respawn Timer progress
4	bool	Is free respawn available
5	uint32	Gold Coin cost required to respawn
6	bool	Is spending Gold Coin for respawn allowed?

```
message RobotRespawnStatus {
  optional bool is_pending_revive = 1;
```

 © 2026 DJI All Rights Reserved

```
  optional uint32 total_revive_progress = 2;
  optional uint32 current_revive_progress = 3;
  optional bool can_revive_for_free = 4;
  optional uint32 gold_coin_cost_for_revive = 5;
  optional bool can_pay_for_Revive = 6;
}
```

2.2.10 RobotStaticStatus

Purpose: Robots' fixed attributes and configuration.

Data ID	Data type	Data purpose
1	uint32	Connection status (0 = not connected, 1 = connected)
2	uint32	Field status (0 = Entered the field, 1 = Not entered)
3	uint32	Alive Status (0 = Unknown, 1 = Alive, 2 = Defeated)
4	uint32	Robot numbering
5	uint32	Robot type
6	uint32	Performance System - Launching Mechanism
7	uint32	Performance System - Chassis
8	uint32	Current level
9	uint32	Maximum HP
10	uint32	Maximum Heat
11	float	Barrel Heat Cooling Rate (per second)
12	uint32	Maximum Power
13	uint32	Maximum Buffer Energy
14	uint32	Maximum Chassis Energy

© 2026 DJI All Rights Reserved

Enumeration of performance system shooter values:

- 1: Cooling-focused
- 2: Burst-focused
- 3: Hero Melee-focused
- 4: Hero Sniper-focused

performance_system_chassis enumeration values:

- 1: HP-focused
- 2: Power-focused
- 3: Hero Melee-focused
- 4: Hero Sniper-focused

```
message RobotStaticStatus {
  optional uint32 connection_state = 1;
  optional uint32 field_state = 2;
  optional uint32 alive_state = 3;
  optional uint32 robot_id = 4;
  optional uint32 robot_type = 5;
  optional uint32 performance_system_shooter = 6;
  optional uint32 performance_system_chassis = 7;
  optional uint32 level = 8;
  optional uint32 max_health = 9;
  optional uint32 max heat = 10;
  optional float heat cooldown rate = 11;
  optional uint32 max_power = 12;
  optional uint32 max_buffer_energy = 13;
  optional uint32 max_chassis_energy = 14;
}
```

2.2.11 RobotsDynamicStatus

Purpose: Real-time data for robots.

Data ID	Data type	Data purpose
1	uint32	Current HP
2	float	Current heat

 © 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
3	float	Previous projectile velocity
4	uint32	Current remaining Chassis energy
5	uint32	Current Buffer Energy
6	uint32	Current Experience Point
7	uint32	Experience still needed to reach the next level up
8	uint32	Total projectiles fired
9	uint32	Remaining Projectile Allowance
10	bool	Is Out-of-Combat status active
11	uint32	Countdown to Out-of-Combat status
12	bool	Is remote HP restoration available
13	bool	Is remote reload available

```
message RobotDynamicStatus {
  optional uint32 current_health = 1;
  optional float current_heat = 2;
  optional float last projectile firing rate = 3;
  optional uint32 current chassis energy = 4;
  optional uint32 current buffer energy = 5;
  optional uint32 current_experience = 6;
  optional uint32 experience_needed_for_upgrade = 7;
  optional uint32 total_projectiles_fired = 8;
  optional uint32 remaining_ammo = 9;
  optional bool is_out_of_combat = 10;
  optional uint32 out_of_combat_countdown = 11;
  optional bool can_be_repaired_remotely = 12;
  optional bool can_remote_ammo = 13;
}
```

© 2026 DJI All Rights Reserved

2.2.12 RobotModuleStatus

Purpose: Operating status of each module in Robots.

Data ID	Data type	Data purpose
1	uint32	Power Management Module status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
2	uint32	RFID module status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
3	uint32	Light Indicator Module status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
4	uint32	17 mm Launching Mechanism status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
5	uint32	42 mm Launching Mechanism status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
6	uint32	UWB Positioning Module status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
7	uint32	Armor Module Status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
8	uint32	Video Transmission Module Status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
9	uint32	Capacitor Module Status (0=Offline, 1=Online)
10	uint32	Main Controller status (0= Offline, 1 = online, 2 = considered Offline due to improper installation)
11	Uint32	Laser detection module status (0 = Offline, 1 = Online, 2 = considered Offline due to improper installation)

```
message RobotModuleStatus {
  optional uint32 power_manager = 1;
  optional uint32 RFID = 2;
  optional uint32 light_strip = 3;
  optional uint32 small_shooter = 4;
  optional uint32 big_shooter = 5;
```

 © 2026 DJI All Rights Reserved

```
  optional uint32 uwb = 6;
  optional uint32 armor = 7;
  optional uint32 video transmission = 8;
  optional uint32 capacitor = 9;
  optional uint32 main_controller = 10;
  optional uint32 laser_detection_module = 11;
}
```

2.2.13 RobotPosition

Purpose: Robots' spatial coordinates and orientation.

Data ID	Data type	Data purpose
1	float	World coordinate X-axis
2	float	World coordinate Y-axis
3	float	World coordinate Z-axis
4	float	The orientation of this robot's Speed Monitor Module, uint: degree, with true north set as 0 degree.
5	uint32	The received coordinates are assigned to the robot: 6/106 refers to the Drone, and 7/107 refers to the Sentry (effective only on the gimbal operator client).

```
message RobotPosition {
  optional float x = 1;
  optional float y = 2;
  optional float z = 3;
  optional float yaw = 4;
}
```

2.2.14 Buff

Purpose: Buff effect information.

Data ID	Data type	Data purpose
1	uint32	Robot ID
2	uint32	Buff type

© 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
3	int32	Buff value (When the buff type enumeration value is 1, 2, 4, or 5, this value is a percentage. For example, if the buff type is a Defense Buff and the buff value is 50, it represents a 50% Defense Buff. When the buff type is 3, this value is a direct value, and a value of x indicates an increase of x/s in heat cooling)
4	uint32	Maximum remaining time for buff
5	uint32	Buff remaining time

Enumeration of buff type values:

- 1: Attack Buff
- 2: Defense Buff/Debuff (vulnerability)
- 3: Barrel Heat Cooling Buff
- 4: Chassis Power Buff
- 5: HP Recovery Buff
- 6: Exchangeable Projectile Allowance
- 7: Terrain Crossing Buff (Preparation) (refers to scenarios where the first half of the Launch Ramp's RFID Interaction Module is detected, but the actual Terrain Crossing buff from the Launch Ramp has not been obtained)

```
message Buff {
  optional uint32 robot_id = 1;
  optional uint32 buff_type = 2;
  optional int32 buff_level = 3;
  optional uint32 buff_max_time = 4;
  optional uint32 buff_left_time = 5;
}
```

2.2.15 PenaltyInfo

Purpose: Penalty information synchronization.

Data ID	Data type	Data purpose
1	uint32	Current penalty type

 © 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
2	uint32	Current penalty duration (seconds)
3	uint32	Current penalty count

Enumeration of penalty_type values:

- 1: Yellow Card
- 2: Dual Yellow Card
- 3: Red Card
- 4: Overpower
- 5: Overheating
- 6: Exceeding firing rate

```
message PenaltyInfo {
  optional uint32 penalty_type = 1;
  optional uint32 penalty_effect_sec = 2;
  optional uint32 total_penalty_num = 3;
}
```

2.2.16 RobotPathPlanInfo

Purpose: Sentry trajectory planning information.

Data ID	Data type	Data purpose
1	uint32	Sentry intention (1 = Attack, 2 = Defense, 3 = Move)
2	uint32	Starting point X coordinate (dm)
3	uint32	Starting point Y coordinate (dm)
4	repeated int32	Array of relative X increments from the starting point (-128 to +127, length 49)
5	repeated int32	Array of relative Y-axis increments from the starting point (-128 to +127, length 49)
6	uint32	Sender ID

© 2026 DJI All Rights Reserved

```
message RobotPathPlanInfo {
  optional uint32 intention = 1;
  optional uint32 start_pos_x = 2;
  optional uint32 start_pos_y = 3;
  repeated int32 offset_x = 4 [packed = true];
  repeated int32 offset_y = 5 [packed = true];
  optional uint32 sender_id = 6;
}
```

2.2.17 MapClickInfo

Purpose: Synchronize information when tapping markers on the map.

Data ID	Data type	Data purpose
1	uint32	Transmission range (0 = specified client, 1 = excluding Sentry, 2 = including Sentry)
2	bytes	List of target robots IDs (fixed at 7 bytes; when sending to Own Side Hero and Engineer, Red Team should enter 1, 2, and Blue Team should enter 101, 102, with the remaining digits filled with 0)
3	uint32	Marker type (1 = Attack, 2 = Defense, 3 = Alert, 4 = Custom)
4	uint32	ID of the marked opponent robot
5	uint32	Custom icon ASCII code (only valid when the marker type is 4: Custom. Only allows sending official Player's Client-acceptable letters, ranging from (inclusive) "C" to "L", "N", "O", "Q" to "Z")
6	uint32	Marking mode (1 = Map, 2 = Opponent robots
7	float	Map X coordinate
8	float	Map Y coordinate

```
message MapClickInfo {
  optional uint32 is_send_all = 1;
  optional bytes robot_id = 2;
  optional uint32 mode = 3;
  optional uint32 enemy_id = 4;
  optional uint32 ascii = 5;
```

 © 2026 DJI All Rights Reserved

```
  optional uint32 type = 6;
  optional float map_x = 7;
  optional float map_y = 8;
}
```

2.2.18 MapClickCmd

Purpose: Map click marker command.

Data ID	Data type	Data purpose
1	uint32	Transmission range (0 = specified client, 1 = excluding Sentry, 2 = including Sentry)
2	bytes	List of target robots IDs (fixed at 7 bytes; when sending to Own Side Hero and Engineer, Red Team should enter 1, 2, and Blue Team should enter 101, 102, with the remaining digits filled with 0)
3	uint32	Marker type (1 = Attack, 2 = Defense, 3 = Alert, 4 = Custom)
4	uint32	ID of the marked opponent robot
5	uint32	Custom icon ASCII code (only valid when the marker type is 4: Custom. Only allows sending official Player's Client-acceptable letters, ranging from (inclusive) "C" to "L", "N", "O", "Q" to "Z")
6	uint32	Marking mode (1 = Map, 2 = Opponent robots
7	float	Map X coordinate
8	float	Map Y coordinate

```
message MapClickCmd {
  optional uint32 is_send_all = 1;
  optional bytes robot_id = 2;
  optional uint32 mode = 3;
  optional uint32 enemy_id = 4;
  optional uint32 ascii = 5;
  optional uint32 type = 6;
  optional float map_x = 7;
  optional float map_y = 8;
}
```

© 2026 DJI All Rights Reserved

2.2.19 RadarInfoToClient

Purpose: Robot location information transmitted by Radar.

<table>
<tr><th>Data ID</th><th>Data type</th></tr>
<tr><td>1</td><td>repeated uint32
The Radar sends all robot location information and special identification status. Each time transmission is triggered, it sends a data packet with the following structure, arranged in the order of: Opponent's Robot No.1, No.2, No.3, No.4, No.6, No.7; Own side’s Robot No.1, No.2, No.3, No.4, No.6, No.7.
<table>
<tr><th>Data ID</th><th>Data type</th><th>Data purpose</th></tr>
<tr><td>1</td><td>uint32</td><td>Robot position x (cm)</td></tr>
<tr><td>2</td><td>uint32</td><td>Robot position y (cm)</td></tr>
<tr><td>3</td><td>uint32</td><td>Robot special identification (0=No; 1=Yes; 2=Yes, but the target robot's positioning module is currently offline)</td></tr>
</table>
</td></tr>
</table>

```
message RadarInfoToClient {
  repeated uint32 RadarSingleRobotInfo;
}
message RadarSingleRobotInfo
{
  optional uint32 target_pos_x=1;
  optional uint32 target_pos_y=2;
  optional uint32 is_high_light=3;
}
```

2.2.20 CustomByteBlock

Purpose: Custom data stream.

Data ID	Data type	Data purpose
1	bytes	Custom data packet up to 2.4 kbit

```
message CustomByteBlock {
  bytes data = 1;
}
```

 © 2026 DJI All Rights Reserved

2.2.21 AssemblyCommand

Purpose: Engineer assembly command.

Data ID	Data type	Data purpose
1	uint32	Assembly operation type (0 = start exchange, 1 = confirm assembly, 2 = cancel assembly)
2	uint32	Selected assembly difficulty

```
message AssemblyCommand {
  optional uint32 operation = 1;
  optional uint32 difficulty = 2;
}
```

2.2.22 TechCoreMotionStateSync

Purpose: Motion status synchronization for the Tech Core.

Data ID	Data type	Data purpose
1	uint32	The highest difficulty level currently available for assembly selection
2	uint32	Tech Core basic status
3	uint32	Energy unit insertion status: 0 indicates not inserted, 1 indicates inserted
4	uint32	Energy unit translation status. 0 indicates not translated, 1 indicates translated
5	uint32	Energy unit rotation status: 0 for unrotated, 1 for rotated
6	uint32	Opponent tech core status (only valid when Own Side Level 4 assembly is available, 0 means the Opponent is not assembling, 1 means the Opponent is assembling a non-Level 4 Difficulty, and 2 means the Opponent is assembling a Level 4 Difficulty)
7	uint32	Total remaining assembly time for Own Side (only applicable when Own Side is performing a Level 4 assembly)

© 2026 DJI All Rights Reserved

Data ID	Data type	Data purpose
8	uint32	Remaining interval time for each Own Side assembly step (only applies when Own Side is performing a Level 4 assembly)

```
message TechCoreMotionStateSync {
  optional uint32 maximum_difficulty_level = 1;
  optional uint32 basic_state = 2;
  optional uint32 putin_state = 3;
  optional uint32 move_state = 4;
  optional uint32 rotate_state = 5;
  optional uint32 enemy_core_status = 6;
  optional uint32 remain_time_all = 7;
  optional uint32 remain_time_step = 8;
}
```

2.2.23 RobotPerformanceSelectionCommand

Purpose: Ground Robot selects a Performance System or Control mode.

Data ID	Data type	Data purpose
1	uint32	Launching Mechanism Performance System
2	uint32	Chassis Performance System
3	uint32	Sentry Control Mode Selection

Performance System enumeration values can be found in the description of RobotStaticStatus.

sentry_control enumeration values:

- 0 = sentry automatic Control
- 1 = semi-automatic Control for the sentry

```
message RobotPerformanceSelectionCommand {
  optional uint32 shooter = 1;
  optional uint32 chassis = 2;
  optional uint32 sentry_control = 3;
}
```

 © 2026 DJI All Rights Reserved

2.2.24 RobotPerformanceSelectionSync

Purpose: Synchronize performance system status for Infantry/Hero.

Data ID	Data type	Data purpose
1	uint32	Launching Mechanism Performance System
2	uint32	Chassis Performance System
3	uint32	Sentry Control Mode Selection

```
message RobotPerformanceSelectionSync {
  optional uint32 shooter = 1;
  optional uint32 chassis = 2;
  optional uint32 sentry_control = 3;
}
```

2.2.25 CommonCommand

Purpose: multiple common Commands for Robots.

Data ID	Data type	Data purpose
1	uint32	Command type
2	uint32	Parameters corresponding to each Command type

cmd_type enumeration values:

- 1: Exchange 17mm ammuintion (only valid when the remaining Gold Coin balance is sufficient for the Exchange; the parameter value for this Command type must be a multiple of 10)
- 2: Exchange 42mm ammo capacity (valid only when the remaining Gold Coin is sufficient for the Exchange)
- 3: OK Revive (if the Respawn Timer has finished, you will Revive immediately)
- 4: Exchange Instant Respawn (if the conditions for Exchange Instant Respawn are met at this time, Gold Coin will be consumed immediately to Exchange Instant Respawn)
- 5: Remote Exchange Projectile Allowance (if the conditions are met, Gold Coin will be consumed immediately to perform a Remote Exchange Projectile Allowance)
- 6: Remote Exchange for health (if the rules are met at this moment, Gold Coin will be consumed

© 2026 DJI All Rights Reserved

immediately to perform a Remote Exchange for health)

```
message CommonCommand{
  optional uint32 cmd_type = 1;
  optional uint32 param = 2;
}
```

2.2.26 HeroDeployModeEventCommand

Purpose: Hero Deploy Mode command.

Data ID	Data type	Data purpose
1	uint32	Mode (0 = Exit, 1 = Enter)

```
message HeroDeployModeEventCommand{
  optional uint32 mode = 1;
}
```

2.2.27 DeployModeStatusSync

Purpose: Hero Deployment Mode status synchronization.

Data ID	Data type	Data purpose
1	uint32	Current deployment mode status (0 = not deployed, 1 = deployed)

```
message DeployModeStatusSync {
  optional uint32 status = 1;
}
```

2.2.28 RuneActivateCommand

Purpose: Power Rune activation command.

Data ID	Data type	Data purpose
1	uint32	Activate (1 = On)

```
message RuneActivateCommand {
  optional uint32 activate = 1;
}
```

 © 2026 DJI All Rights Reserved

2.2.29 RuneStatusSync

Purpose: Power Rune status synchronization.

Data ID	Data type	Data purpose
1	uint32	Enumeration of current Power Rune statuses
2	uint32	Current number of activated light arms
3	float	Average rings (only valid when the Large Power Rune is active; in all other states, this value is 0)

Enumeration of rune_status values:

- 1: Not Activated
- 2: Activating
- 3: Activated

```
message RuneStatusSync {
  optional uint32 rune_status = 1;
  optional uint32 activated_arms = 2;
  optional float average_rings = 3;
}
```

2.2.30 SentryStatusSync

Purpose: Sentry mode and weakened state.

Data ID	Data type	Data purpose
1	uint32	Mode ID (1 for offensive, 2 for defensive , 3 for mobile)
2	bool	Is weakened
3	bool	Is strengthened

```
message SentryStatusSync {
  optional uint32 posture_id = 1;
  optional bool is_weakened = 2;
  optional bool is_powered = 3;
}
```

© 2026 DJI All Rights Reserved

2.2.31 DartCommand

Purpose: Dart control command.

Data ID	Data type	Data purpose
1	uint32	Target ID (1: Outpost, 2: Base fixed point, 3: Base randomly assigned fixed point, 4: Base randomly assigned moving point, 5: Base terminal moving point)
2	bool	Gate switch
3	bool	Whether to OK the Launch (default is 0, 1 means OK to Launch)

```
message DartCommand {
  optional uint32 target_id = 1;
  optional bool open = 2;
  optional bool launch_confirm = 3;
}
```

2.2.32 DartSelectControlStatusSync

Purpose: Dart target selection status synchronization.

Data ID	Data type	Data purpose
1	uint32	Target ID
2	uint32	Gate status (0: closed, 1: opening, 2: open)

```
message DartSelectControlStatusSync {
  optional uint32 target_id = 1;
  optional uint open = 2;
}
```

2.2.33 SentryCtrlCommand

Purpose: Sentry Control command request.

Data ID	Data type	Data purpose
1	uint32	Command ID (0 = invalid)

 © 2026 DJI All Rights Reserved

Enumeration of command code :

- 1: Reload at Restoration Zone
- 3: Remote reload Remote reload
- 4: Remote HP recovery
- 5: Confirm respawn
- 6: Confirm to spend Gold Coin for respawn
- 7: Switch to offensive mode
- 8: Switch to defensive mode
- 9: Switch to mobile mode
- 10: Switch to strengthened offensive mode
- 11: Switch to strengthened defensive mode
- 12: Switch to strengthened mobile mode

```
message SentryCtrlCommand {
  optional uint32 command_id = 1;
}
```

2.2.34 SentryCtrlResult

Purpose: Feedback on Sentry control command.

Data ID	Data type	Data purpose
1	uint32	Corresponding Command ID
2	uint32	Result code

Enumeration of result_code values:

- 0: Success
- Miscellaneous: Failed

```
message SentryCtrlResult {
  optional uint32 command_id = 1;
  optional uint32 result_code = 2;
}
```

© 2026 DJI All Rights Reserved

2.2.35 AirSupportCommand

Purpose: Air support command.

Data ID	Data type	Data purpose
1	uint32	Command Type

Enumeration of command_id values:

- 0: Cancel air support
- 1: Request air support for free
- 2: Request air support using Gold Coin (free duration will still be used first)
- 3: Spend gold coins to remove the countermeasures (only effective under the circumstances described by the rules)

```
message AirSupportCommand {
  optional uint32 command_id = 1;
}
```

2.2.36 AirSupportStatusSync

Purpose: Air support status feedback.

Data ID	Data type	Data purpose
1	uint32	Air Support Status
2	uint32	Remaining time for free air support
3	uint32	Spent Gold Coins for paid air support
4	uint32	Whether the current laser detection module is detecting irradiation (0: not irradiated, 1: irradiated)
5	uint32	Drone countermeasure status

Enumeration of AirSupport_Status values:

- 0: Air support not activated
- 1: Air support in progress

Enumeration of shooter_status:

 © 2026 DJI All Rights Reserved

- 0: Launching Mechanism locked due to Radar countermeasure and cannot be unlocked
- 1: Launching Mechanism operating normally, not being countered
- 2: Launching Mechanism locked due to Radar countermeasure and can be unlocked by spending Gold Coin

```
message AirSupportStatusSync {
  optional uint32 airsupport_status = 1;
  optional uint32 left_time = 2;
  optional uint32 cost_gold_coin = 3;
  optional uint32 is_being_targeted = 4;
  optional uint32 shooter_status = 5;
}
```

© 2026 DJI All Rights Reserved

Appendix I: CRC check code example

```
//crc8 generator polynomial:G(x)=x8+x5+x4+1
const unsigned char CRC8_INIT = 0xff;
const unsigned char CRC8_TAB[256] =
{
0x00, 0x5e, 0xbc, 0xe2, 0x61, 0x3f, 0xdd, 0x83, 0xc2, 0x9c, 0x7e, 0x20, 0xa3, 0xfd, 0x1f, 0x41,
0x9d, 0xc3, 0x21, 0x7f, 0xfc, 0xa2, 0x40, 0x1e, 0x5f, 0x01, 0xe3, 0xbd, 0x3e, 0x60, 0x82, 0xdc, 0x23,
0x7d, 0x9f, 0xc1, 0x42, 0x1c, 0xfe, 0xa0, 0xe1, 0xbf, 0x5d, 0x03, 0x80, 0xde, 0x3c, 0x62, 0xbe, 0xe0,
0x02, 0x5c, 0xdf, 0x81, 0x63, 0x3d, 0x7c, 0x22, 0xc0, 0x9e, 0x1d, 0x43, 0xa1, 0xff, 0x46, 0x18, 0xfa,
0xa4, 0x27, 0x79, 0x9b, 0xc5, 0x84, 0xda, 0x38, 0x66, 0xe5, 0xbb, 0x59, 0x07, 0xdb, 0x85, 0x67,
0x39, 0xba, 0xe4, 0x06, 0x58, 0x19, 0x47, 0xa5, 0xfb, 0x78, 0x26, 0xc4, 0x9a, 0x65, 0x3b, 0xd9, 0x87,
0x04, 0x5a, 0xb8, 0xe6, 0xa7, 0xf9, 0x1b, 0x45, 0xc6, 0x98, 0x7a, 0x24, 0xf8, 0xa6, 0x44, 0x1a, 0x99,
0xc7, 0x25, 0x7b, 0x3a, 0x64, 0x86, 0xd8, 0x5b, 0x05, 0xe7, 0xb9,
0x8c, 0xd2, 0x30, 0x6e, 0xed, 0xb3, 0x51, 0x0f, 0x4e, 0x10, 0xf2, 0xac, 0x2f, 0x71, 0x93, 0xcd, 0x11,
0x4f, 0xad, 0xf3, 0x70, 0x2e, 0xcc, 0x92, 0xd3, 0x8d, 0x6f, 0x31, 0xb2, 0xec, 0x0e, 0x50, 0xaf, 0xf1,
0x13, 0x4d, 0xce, 0x90, 0x72, 0x2c, 0x6d, 0x33, 0xd1, 0x8f, 0x0c, 0x52, 0xb0, 0xee, 0x32, 0x6c, 0x8e,
0xd0, 0x53, 0x0d, 0xef, 0xb1, 0xf0, 0xae, 0x4c, 0x12, 0x91, 0xcf, 0x2d, 0x73, 0xca, 0x94, 0x76, 0x28,
0xab, 0xf5, 0x17, 0x49, 0x08, 0x56, 0xb4, 0xea, 0x69, 0x37, 0xd5, 0x8b, 0x57, 0x09, 0xeb, 0xb5,
0x36, 0x68, 0x8a, 0xd4, 0x95, 0xcb, 0x29, 0x77, 0xf4, 0xaa, 0x48, 0x16, 0xe9, 0xb7, 0x55, 0x0b, 0x88,
0xd6, 0x34, 0x6a, 0x2b, 0x75, 0x97, 0xc9, 0x4a, 0x14, 0xf6, 0xa8,
0x74, 0x2a, 0xc8, 0x96, 0x15, 0x4b, 0xa9, 0xf7, 0xb6, 0xe8, 0x0a, 0x54, 0xd7, 0x89, 0x6b, 0x35,
};
unsigned char Get_CRC8_Check_Sum(unsigned char *pchMessage,unsigned int
dwLength,unsigned char ucCRC8)
{
unsigned char ucIndex;
while (dwLength--)
{
ucIndex = ucCRC8^(*pchMessage++);
ucCRC8 = CRC8_TAB[ucIndex];
}
return(ucCRC8);
}
/*
** Descriptions: CRC8 Verify function
** Input: Data to Verify,Stream length = Data + checksum
** Output: True or False (CRC Verify Result)
*/
unsigned int Verify_CRC8_Check_Sum(unsigned char *pchMessage, unsigned int dwLength)
{
```

 © 2026 DJI All Rights Reserved

```
unsigned char ucExpected = 0;
if ((pchMessage == 0) || (dwLength <= 2)) return 0;
ucExpected = Get_CRC8_Check_Sum (pchMessage, dwLength-1, CRC8_INIT);
return ( ucExpected == pchMessage[dwLength-1] );
}
/*
** Descriptions: append CRC8 to the end of data
** Input: Data to CRC and append,Stream length = Data + checksum
** Output: True or False (CRC Verify Result)
*/
void Append_CRC8_Check_Sum(unsigned char *pchMessage, unsigned int dwLength)
{
unsigned char ucCRC = 0;
if ((pchMessage == 0) || (dwLength <= 2)) return;
ucCRC = Get_CRC8_Check_Sum ( (unsigned char *)pchMessage, dwLength-1, CRC8_INIT);
pchMessage[dwLength-1] = ucCRC;
}

uint16_t CRC_INIT = 0xffff;
const uint16_t wCRC_Table[256] =
{
0x0000, 0x1189, 0x2312, 0x329b, 0x4624, 0x57ad, 0x6536, 0x74bf,
0x8c48, 0x9dc1, 0xaf5a, 0xbed3, 0xca6c, 0xdbe5, 0xe97e, 0xf8f7,
0x1081, 0x0108, 0x3393, 0x221a, 0x56a5, 0x472c, 0x75b7, 0x643e,
0x9cc9, 0x8d40, 0xbfdb, 0xae52, 0xdaed, 0xcb64, 0xf9ff, 0xe876,
0x2102, 0x308b, 0x0210, 0x1399, 0x6726, 0x76af, 0x4434, 0x55bd,
0xad4a, 0xbcc3, 0x8e58, 0x9fd1, 0xeb6e, 0xfae7, 0xc87c, 0xd9f5,
0x3183, 0x200a, 0x1291, 0x0318, 0x77a7, 0x662e, 0x54b5, 0x453c,
0xbdcb, 0xac42, 0x9ed9, 0x8f50, 0xfbef, 0xea66, 0xd8fd, 0xc974,
0x4204, 0x538d, 0x6116, 0x709f, 0x0420, 0x15a9, 0x2732, 0x36bb,
0xce4c, 0xdfc5, 0xed5e, 0xfcd7, 0x8868, 0x99e1, 0xab7a, 0xbaf3,
0x5285, 0x430c, 0x7197, 0x601e, 0x14a1, 0x0528, 0x37b3, 0x263a,
0xdecd, 0xcf44, 0xfddf, 0xec56, 0x98e9, 0x8960, 0xbbfb, 0xaa72,
0x6306, 0x728f, 0x4014, 0x519d, 0x2522, 0x34ab, 0x0630, 0x17b9,
0xef4e, 0xfec7, 0xcc5c, 0xddd5, 0xa96a, 0xb8e3, 0x8a78, 0x9bf1,
0x7387, 0x620e, 0x5095, 0x411c, 0x35a3, 0x242a, 0x16b1, 0x0738,
0xffcf, 0xee46, 0xdcdd, 0xcd54, 0xb9eb, 0xa862, 0x9af9, 0x8b70,
0x8408, 0x9581, 0xa71a, 0xb693, 0xc22c, 0xd3a5, 0xe13e, 0xf0b7,
0x0840, 0x19c9, 0x2b52, 0x3adb, 0x4e64, 0x5fed, 0x6d76, 0x7cff,
```

© 2026 DJI All Rights Reserved

```
0x9489, 0x8500, 0xb79b, 0xa612, 0xd2ad, 0xc324, 0xf1bf, 0xe036,
0x18c1, 0x0948, 0x3bd3, 0x2a5a, 0x5ee5, 0x4f6c, 0x7df7, 0x6c7e,
0xa50a, 0xb483, 0x8618, 0x9791, 0xe32e, 0xf2a7, 0xc03c, 0xd1b5,
0x2942, 0x38cb, 0x0a50, 0x1bd9, 0x6f66, 0x7eef, 0x4c74, 0x5dfd,
0xb58b, 0xa402, 0x9699, 0x8710, 0xf3af, 0xe226, 0xd0bd, 0xc134,
0x39c3, 0x284a, 0x1ad1, 0x0b58, 0x7fe7, 0x6e6e, 0x5cf5, 0x4d7c,
0xc60c, 0xd785, 0xe51e, 0xf497, 0x8028, 0x91a1, 0xa33a, 0xb2b3,
0x4a44, 0x5bcd, 0x6956, 0x78df, 0x0c60, 0x1de9, 0x2f72, 0x3efb,
0xd68d, 0xc704, 0xf59f, 0xe416, 0x90a9, 0x8120, 0xb3bb, 0xa232,
0x5ac5, 0x4b4c, 0x79d7, 0x685e, 0x1ce1, 0x0d68, 0x3ff3, 0x2e7a,
0xe70e, 0xf687, 0xc41c, 0xd595, 0xa12a, 0xb0a3, 0x8238, 0x93b1,
0x6b46, 0x7acf, 0x4854, 0x59dd, 0x2d62, 0x3ceb, 0x0e70, 0x1ff9,
0xf78f, 0xe606, 0xd49d, 0xc514, 0xb1ab, 0xa022, 0x92b9, 0x8330,
0x7bc7, 0x6a4e, 0x58d5, 0x495c, 0x3de3, 0x2c6a, 0x1ef1, 0x0f78
};
/*
** Descriptions: CRC16 checksum function
** Input: Data to check,Stream length, initialized checksum
** Output: CRC checksum
*/
uint16_t Get_CRC16_Check_Sum(uint8_t *pchMessage,uint32_t dwLength,uint16_t wCRC)
{
Uint8_t chData;
if (pchMessage == NULL)
{
return 0xFFFF;
}
while(dwLength--)
{
chData = *pchMessage++;
(wCRC) = ((uint16_t)(wCRC) >> 8) ^ wCRC_Table[((uint16_t)(wCRC) ^ (uint16_t)(chData)) & 0x00ff];
}
return wCRC;
}

/*
** Descriptions: CRC16 Verify function
** Input: Data to Verify,Stream length = Data + checksum
** Output: True or False (CRC Verify Result)
```

 © 2026 DJI All Rights Reserved

```
*/
uint32_t Verify_CRC16_Check_Sum(uint8_t *pchMessage, uint32_t dwLength)
{
uint16_t wExpected = 0;
if ((pchMessage == NULL) || (dwLength <= 2))
{
return __FALSE;
}
wExpected = Get_CRC16_Check_Sum ( pchMessage, dwLength - 2, CRC_INIT);
return ((wExpected & 0xff) == pchMessage[dwLength - 2] && ((wExpected >> 8) & 0xff) ==
pchMessage[dwLength - 1]);
}

/*
** Descriptions: append CRC16 to the end of data
** Input: Data to CRC and append,Stream length = Data + checksum
** Output: True or False (CRC Verify Result)
*/
void Append_CRC16_Check_Sum(uint8_t * pchMessage,uint32_t dwLength)
{
uint16_t wCRC = 0;
if ((pchMessage == NULL) || (dwLength <= 2))
{
return;
}
wCRC = Get_CRC16_Check_Sum ( (U8 *)pchMessage, dwLength-2, CRC_INIT );
pchMessage[dwLength-2] = (U8)(wCRC & 0x00ff);
pchMessage[dwLength-1] = (U8)((wCRC >> 8)& 0x00ff);
```

© 2026 DJI All Rights Reserved

Appendix II: Instruction for ID numbers

Robot ID numbers are as follows:

- 1: Red Team Hero
- 2: Red Team Engineer
- 3/4/5: Red Team Infantry robots (corresponding to robot IDs 3–5
- 6: Red Team Drone
- 7: Red Team Sentry
- 8: Red Team Dart
- 9: Red Team Radar
- 10: Red Team Base
- 11: Red Team Outpost
- 101: Blue Team Hero
- 102: Blue Team Engineer
- 103/104/105: Blue Team Infantry robots (corresponding to robot IDs 3–5
- 106: Blue Team Drone
- 107: Blue Team Sentry
- 108: Blue Team Dart
- 109: Blue Team Radar
- 110: Blue Team Base
- 111: Blue Team Outpost
- Player's Client ID is as follows:
- 0x0101: Red Team Hero Player's Client
- 0x0102: Red Team Engineer Player's Client
- 0x0103/0x0104/0x0105: Red Team Infantry Player's Client (corresponding to Robots ID 3–5)
- 0x0106: Red Team Drone Player's Client
- 0x016A: Blue Team Drone Player's Client
- 0x0165: Blue Team Hero Player's Client

 © 2026 DJI All Rights Reserved

- 0x0166: Blue Team Engineer Player's Client
- 0x0167/0x0168/0x0169: Blue Team Infantry Player's Client (corresponding to Robots with ID 3 to 5)
- 0x8080: Referee system Server (used for sentry and Radar Decision Commands)

© 2026 DJI All Rights Reserved

Appendix III: Sample communication code for custom client

```
using MQTTnet;
using MQTTnet.Client;
using MQTTnet.Diagnostics;
using MQTTnet.Protocol;
using MQTTnet.Server;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;
using UnityEngine;

/// <summary>
/// MQTT Client
/// </summary>
public class MyMqttClient
{
    private IMqttClient m_MqttClient = null;
    private string m_ClientID;

    /// <summary>
    /// Create a client and connect to the server
    /// </summary>
    /// <param name="clientID"></param>
    /// <param name="ip"></param>
    /// <param name="port"></param>
    public MyMqttClient(string clientID, string ip = "127.0.0.1", int port = 3333)
    {
        m_ClientID = clientID;
        // Client options builder
        var options = new MqttClientOptionsBuilder()
        .WithClientId(m_ClientID)
        .WithTcpServer(ip, port)
        .Build();
        // Create client
        m_MqttClient = new MqttFactory().CreateMqttClient();
        // Monitor client connect or disconnect complete
        m_MqttClient.ConnectedAsync += ClientConnected;
        m_MqttClient.DisconnectedAsync += ClientDisConnected;
        // Client received message
        m_MqttClient.ApplicationMessageReceivedAsync += ReceiveMsg;
        // Connect to the server
```

 © 2026 DJI All Rights Reserved

```
        m_MqttClient.ConnectAsync(options);
    }

    /// <summary>
    /// Message received
    /// </summary>
    /// <param name="args"></param>
    /// <returns></returns>
    private Task ReceiveMsg(MqttApplicationMessageReceivedEventArgs args)
    {
     Debuger.Log(DebugLogEnum.CustomClient, $"recv topic: {args.ApplicationMessage.Topic}");
//Message post appears to avoid message blockage
        MqttSubProtoEvent ev = new MqttSubProtoEvent();
        ev.topic = args.ApplicationMessage.Topic;
        ev.data = args.ApplicationMessage.Payload;
        ThreadPool.CustomClientLoop.PostEvent(ev);
return Task.CompletedTask;
    }

    /// <summary>
    /// Disconnection completed
    /// </summary>
    /// <param name="args"></param>
    /// <returns></returns>
    private Task ClientDisConnected(MqttClientDisconnectedEventArgs args)
    {
        return Task.CompletedTask;
    }

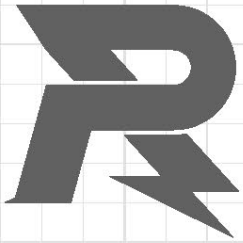
    /// <summary>
    /// Connection completed
    /// </summary>
    /// <param name="args"></param>
    /// <returns></returns>ReceiveMsg
    private Task ClientConnected(MqttClientConnectedEventArgs args)
    {
       Subscribe("KeyboardMouseControl");
        Subscribe("CustomControl");
return Task.CompletedTask;
    }

    /// <summary>
    /// Message release
    /// </summary>
    public void PublishMsg(string topic, string message, MqttQualityOfServiceLevel level =
MqttQualityOfServiceLevel.ExactlyOnce, bool isRetain = false)
```

© 2026 DJI All Rights Reserved

```
    {
        long unixUs = DateTime.UtcNow.Ticks / 10; // microsecond-level
        m_MqttClient.PublishStringAsync(topic, message, level, isRetain);
    }
/// <summary>
    /// Message release
    /// </summary>
    public void PublishBytesMsg(string topic, byte[] message, MqttQualityOfServiceLevel level =
MqttQualityOfServiceLevel.ExactlyOnce, bool isRetain = false)
    {
        long unixUs = DateTime.UtcNow.Ticks / 10; // microsecond-level
        m_MqttClient.PublishBinaryAsync(topic, message, level, isRetain);
    }
    /// <summary>
    /// Subscribe topic
    /// </summary>
    public void Subscribe(string topic)
    {
        m_MqttClient.SubscribeAsync(new MqttTopicFilterBuilder().WithTopic(topic).Build());
    }
}
```

 © 2026 DJI All Rights Reserved



E-mail: robomaster@dji.com
Forum: https://bbs.robomaster.com
Website: https://www.robomaster.com
Tel: +86 0755-84357613 (GTC+8, 10:30AM-7:30PM, Monday to Friday)
Address: T2, 22F, DJI Sky City, No. 55 Xianyuan Road, Nanshan District, Shenzhen, China